
Learning Interpretable Options by Identifying Reward Diffusion Bottlenecks in Reinforcement Learning

Yiming Fei^{1 2} Lang Qin¹ Rui Yan³ Huajin Tang^{1 2}

Abstract

Bottleneck states, which connect distinct regions of the state space, provide a principled and interpretable basis for constructing temporal abstractions in Hierarchical Reinforcement Learning (HRL). However, existing bottleneck identification methods primarily rely on topological analysis of the state-transition graph, limiting their scalability to high-dimensional or continuous domains. To address this challenge, we introduce Value Power Strength (VPS), a value function-based metric inspired by the analogy between the Bellman equation and Kirchhoff’s current law, to quantify bottleneck property via the diffusion of reward in Markov Decision Processes (MDPs). VPS is estimated efficiently using value functions learned from random reward signals and captures reward diffusion bottlenecks in both discrete and continuous state spaces. Leveraging VPS, we design options that guide agents toward or away from bottleneck regions. Experiments on tabular domains, continuous-control, and Atari 2600 games show that VPS identifies semantically meaningful bottlenecks, while the learned options improve exploration.

1. Introduction

Sparse rewards and long horizons motivate HRL, which uses action abstractions or subgoals to reduce exploration complexity (Sutton & Barto, 1998; Ecoffet et al., 2021; Pateria et al., 2021; Nachum et al., 2019). Hierarchical agents have shown gains across navigation, manipulation,

and visual control (Nachum et al., 2018; Park et al., 2023; Eysenbach et al., 2019; Bagaria et al., 2023).

Options are temporally extended policies with initiation and termination conditions (Sutton et al., 1999). Option discovery either exploits graph/topological or spectral structure to select subgoals (Menache et al., 2002; Şimşek et al., 2005; Machado et al., 2017; Klissarov & Machado, 2023) or optimizes objectives such as planning and cover time (Solway et al., 2014; Jinnai et al., 2019a;b; Ivanov et al., 2025).

Bottleneck states—bridges between weakly coupled regions of the state space—have clear semantics and are natural intrinsic targets for option design. Existing methods usually cast the MDP as a graph and score states with topological criteria such as betweenness centrality or cut-based measures (Freeman, 1977; Newman, 2005; Brandes, 2001; Shi & Malik, 2000). Although effective in tabular settings, they require explicit graph construction and compute scores outside the value-learning loop, limiting scalability and integration with function approximation.

MDP value functions can be viewed as potentials governed by Poisson-type equations, graph Laplacians, and random walks (Mahadevan & Maggioni, 2007; Machado et al., 2017; Wang et al., 2021). Circuit theory similarly relates currents, potentials, and power dissipation on weighted graphs (Doyle & Snell, 1984; Chung, 1997; Chandra et al., 1989; Tetali, 1991), motivating value-based primitives for option discovery.

We exploit this correspondence by mapping node power to VPS, a graph-free measure of reward-diffusion bottlenecks. Random rewards induce value functions whose VPS fields provide intrinsic potentials for options that steer toward or away from bottlenecks. Because VPS relies on value estimation, it extends naturally to function approximation.

Our main contributions are: (1) we propose VPS, a value function-based metric for state-level bottlenecks without explicit graph analysis; (2) we derive estimation procedures for discrete and continuous state spaces and prove almost sure convergence to the true VPS under mild assumptions; and (3) we design a VPS-based option discovery scheme that yields semantically meaningful, interpretable temporally extended actions and improves exploration in tabular and

¹College of Computer Science and Technology, Zhejiang University, Hangzhou, China ²The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China ³College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China. Correspondence to: Huajin Tang <htang@zju.edu.cn>.

continuous-control domains, while high-dimensional visual experiments validate VPS-based bottleneck identification.

2. Related Work

VPS is related to Laplacian-based representation learning, where value functions can be viewed as potentials on the state-transition graph and their Dirichlet energy characterizes graph structure. Early proto-value functions (PVFs) use Laplacian eigenvectors as basis functions (Mahadevan, 2005; Mahadevan & Maggioni, 2007). Eigenoptions construct options from Laplacian eigenvectors to follow globally smooth directions (Machado et al., 2017), and later work connects eigenoptions to successor representations (Machado et al., 2018; 2023). Recent advances scale Laplacian learning to high-dimensional settings (Wu et al., 2019; Gomez et al., 2024) and study broader spectral operator learning (Touati et al., 2023; Ryu et al., 2025), enabling on-line spectral option discovery (Klissarov & Machado, 2023) and improved credit assignment (Kotamreddy & Machado, 2025).

Among these spectral option-discovery methods, Eigenoptions are the most direct point of comparison for VPS. A key difference is that eigenoptions are not bottleneck-specific: as the number of eigenvectors (hence options) increases, the extrema of different eigenvectors typically spread across the graph, yielding heterogeneous targets (e.g., room centers, edges, and sometimes passages). Although the Fiedler vector may align with a dominant coarse bottleneck, bottlenecks are generally multi-scale and non-unique, and it is unclear a priori which eigenvectors correspond to bottlenecks at which scales. VPS instead aggregates value-based quantities induced by multiple random rewards into a single bottleneck score, so even with many options the semantics remain anchored to bottleneck states.

VPS also connects to auxiliary-task methods that learn many value predictions from random cumulants (Lyle et al., 2021; Zheng et al., 2021; Ramesh et al., 2022). Proto-Value Networks scale this idea to many auxiliary heads, where the span of random-cumulant value predictions approaches leading successor/Laplacian eigenspaces (Farebrother et al., 2023). Unlike these approaches that use auxiliary values primarily as features, we use random rewards diagnostically to construct a bottleneck-oriented scalar (VPS) and derive options that explicitly steer toward or away from bottlenecks.

3. Preliminaries

RL focuses on training agents to make sequential decisions by interacting with an environment. The process is commonly modeled as an MDP (Puterman, 1994), defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$. At each time step t , the

agent at state $S_t \in \mathcal{S}$ takes an action $A_t \in \mathcal{A}$ and the next state $S_{t+1} \in \mathcal{S}$ is determined by the transition probability kernel $\mathcal{P}(s'|s, a) = \Pr(S_{t+1} = s' | S_t = s, A_t = a)$. Then the environment generates an immediate reward R_{t+1} . The discounted return is the random variable $G_t := \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$, where $\gamma \in [0, 1)$ is a discount factor, and the objective is to learn a policy that maximizes its expectation.

3.1. Bellman Expectation Equation.

Value function-based RL methods (Mnih et al., 2015; Hessel et al., 2018) aim to estimate the value function, which represents the expected return starting from a given state or state-action pair. The state-value function is defined as $V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$. For a stationary policy π , the state-value function satisfies

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) [r(s, a, s') + \gamma V_\pi(s')], \quad (1)$$

which is also known as the Bellman expectation equation.

For finite $\mathcal{S} = \{s_1, \dots, s_{|\mathcal{S}|}\}$, let V_π and r_π collect state values and expected rewards, and define $[P_\pi]_{ij} = \sum_{a \in \mathcal{A}} \pi(a|s_i) \mathcal{P}(s_j | s_i, a)$. Then (1) becomes

$$\mathcal{T}V_\pi = r_\pi, \quad (2)$$

where $\mathcal{T} = I - \gamma P_\pi$ is the linear Bellman equation operator induced by π . For $\gamma < 1$, the closed-form solution is obtained as

$$V_\pi = (I - \gamma P_\pi)^{-1} r_\pi. \quad (3)$$

3.2. Random Rewards via QR Decomposition

For tabular implementation, we generate a set of diverse random rewards by orthogonalizing Gaussian samples. Let $n = |\mathcal{S}|$, choose $1 \leq k \leq n$, and sample a reward matrix $Z \in \mathbb{R}^{n \times k}$ with i.i.d. entries $Z_{si} \sim \mathcal{N}(0, 1)$. We then apply a reduced QR decomposition

$$Z = HC, \quad H^\top H = I_k, \quad (4)$$

where $H, Z \in \mathbb{R}^{n \times k}$ and the upper-triangular factor $C \in \mathbb{R}^{k \times k}$. We use the i -th column of H as the i -th random reward $r^{(i)}(s) = H_{si}$. This QR-based construction preserves the rotational invariance of Gaussian sampling (hence no preferred direction in expectation) while guaranteeing within-batch orthogonality and reducing redundant, highly correlated rewards (Mezzadri, 2007).

3.3. Random Rewards via Random Fourier Features

In high-dimensional or continuous spaces, we construct random rewards using Random Fourier Features (RFF) (Rahimi & Recht, 2007) on the learned representation $z_\omega(s)$. Let

$k(z, z') = \exp(-\|z - z'\|^2 / (2\ell^2))$ be the RBF kernel with length-scale $\ell > 0$. We sample

$$\mathbf{w}_j \sim \mathcal{N}(0, \ell^{-2} I_d), \quad b_j \sim \text{Uniform}(0, 2\pi),$$

and define the K -dimensional randomized feature map

$$\zeta_j(s) = \sqrt{\frac{2}{K}} \cos(\mathbf{w}_j^\top z_\omega(s) + b_j), \quad j = 1, \dots, K.$$

The m -th random reward is a random linear combination of these features,

$$r^{(m)}(s) = \sum_{j=1}^K a_j^{(m)} \zeta_j(s), \quad a_j^{(m)} \sim \mathcal{N}(0, 1) \text{ i.i.d.} \quad (5)$$

As $K \rightarrow \infty$, the finite-dimensional distributions of $r^{(m)}$ converge to those of a zero-mean Gaussian process $\mathcal{GP}(0, k)$, while its covariance converges to the specified kernel k .

3.4. Laplacian of State Transition Graphs.

The state transition graph of an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ provides a structural representation of the environment by modeling the transitions between states as a graph (Şimşek & Barto, 2008). Formally, a state transition graph $\mathcal{G} = (V, E, W)$ is a weighted directed graph where the vertices $V = \mathcal{S}$ correspond to the set of states and the directed edges $E \subseteq \mathcal{S} \times \mathcal{S}$ represent corresponding state transitions. An edge $(s, s') \in E$ exists if there exists an action $a \in \mathcal{A}$ such that the transition probability $\mathcal{P}(s'|s, a) > 0$. The weight function $W : E \rightarrow \mathbb{R}^+$ assigns each edge a positive value that captures the possibility of the corresponding transition.

For an MDP with a finite state set, we embed its transition dynamics in a weighted graph whose adjacency matrix is $W = [W_{ss'}]_{s, s' \in \mathcal{S}}$ and whose diagonal degree matrix is

$$D_W = \text{diag}(d_W(s_1), \dots, d_W(s_{|\mathcal{S}|})), \quad d_W(s) = \sum_{s'} W_{ss'}. \quad (6)$$

For the Laplacian construction, W denotes an undirected weighted graph, or the symmetrized version of the directed transition graph above. The topological structure of the graph is then characterized by the random-walk Laplacian

$$L_{\text{rw}} = I - D_W^{-1} W, \quad (7)$$

whose eigenvectors associated with the smallest (non-zero) eigenvalues minimize the discrete Dirichlet energy and therefore vary smoothly within highly connected regions while changing sharply across narrow bridges (von Luxburg, 2007; Belkin & Niyogi, 2003).

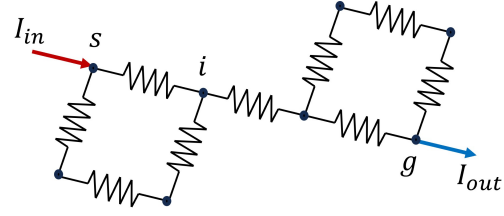


Figure 1. Single-source single-sink resistance network

3.5. Option Framework.

The option framework, introduced by (Sutton et al., 1999), extends RL by enabling agents to perform temporally extended actions, facilitating hierarchical decision-making. An option is defined as a tuple $\mathcal{O} = (\mathcal{I}_o, \pi_o, \beta_o)$, where $\mathcal{I}_o \subseteq \mathcal{S}$ is the initiation set specifying where the option can start, $\pi_o : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is the intra-option policy governing behavior during the option’s execution, and $\beta_o : \mathcal{S} \rightarrow [0, 1]$ is the termination condition defining the probability of ending the option at each state. Agents execute options by selecting an option, following its policy, and terminating with β_o .

4. Option Discovery via Value Power Strength

Unlike traditional graph-based methods that compute bottleneck metrics with transition graphs, our approach leverages the structural analogy between the Bellman equation and Kirchhoff’s current law to propose *Value Power Strength* (VPS)—a value function-based bottleneck metric. VPS enables the construction of intrinsic rewards that guide option learning toward or away from bottleneck regions.

4.1. Value Power Strength of States

Consider the single-source single-sink resistance network illustrated in Figure 1. Current flow through a node is a natural bottleneck score (Newman, 2005):

$$I_i = \frac{1}{2} \sum_j W_{ij} |U_i - U_j|, \quad i \notin \{s, g\}. \quad (8)$$

where $W_{ij} = 1/R_{ij}$ is the edge conductance, U_i is the electric potential, and the factor $1/2$ converts the total absolute incident flow into through-current because flow conservation makes the inflow and outflow contributions equal at an interior node. The source and sink currents are defined separately as $I_s = I_g = I_{in}$, so node current measures the share of the overall charge-transport task borne by each vertex. Bottlenecks can also be viewed through energy conversion: if the network is regarded as a heater, nodes with larger power dissipation bear a larger portion of the transport-induced energy-conversion load. We therefore use node power as a uniform, differentiable bottleneck quantity:

$$\mathcal{P}_i^{\text{elec}} = \frac{1}{2} \sum_j W_{ij} (U_i - U_j)^2. \quad (9)$$

Node power locally decomposes the graph Dirichlet energy (Chung, 1997):

$$\mathcal{E}(U) = \frac{1}{2} \sum_{i,j} W_{ij} (U_i - U_j)^2 = U^\top L_c U = \sum_i \mathcal{P}_i^{\text{elec}}, \quad (10)$$

where L_c is the combinatorial Laplacian. Large $\mathcal{P}_i^{\text{elec}}$ therefore marks where energy concentrates.

To avoid explicit state-transition graphs, we map node power to RL through the Bellman–Kirchhoff analogy. Kirchhoff’s law is

$$L_c U = I, \quad (11)$$

where U and I are node-potential and external-current vectors. To make the relation to (2) precise, write

$$\mathcal{T} = I - \gamma P_\pi = (1 - \gamma)I + \gamma L_{\text{rw}}. \quad (12)$$

For a reversible policy-induced chain with stationary distribution $\mathbf{d}$, let $D_d = \text{diag}(\mathbf{d})$ and $W = D_d P_\pi$. Detailed balance makes W symmetric, its row sums equal $\mathbf{d}$, and $L_c = D_d - W = D_d L_{\text{rw}}$. Multiplying the Bellman system by D_d gives

$$[(1 - \gamma)D_d + \gamma L_c] V_\pi = D_d r_\pi. \quad (13)$$

Thus discounting contributes the leakage (or grounding) term $(1 - \gamma)D_d$ rather than making the Bellman operator identical to L_c . This system is a discrete analogue of the Poisson equation

$$-\nabla^2 \Phi = \rho, \quad (14)$$

where Φ is a potential and ρ is a source. Thus, electric potential $U(s)$ corresponds to value $V(s)$, and external current corresponds to the stationary-weighted reward; Table 1 summarizes this correspondence.

Mapping node power through this analogy yields the following degree-normalized value-based bottleneck metric. The coefficient $1/2$ is omitted since it is shared by all states.

Definition 1 (Value Power Strength). Consider an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ with a stationary policy π . The *Value Power Strength* (VPS) of state $s \in \mathcal{S}$ is defined as the expected squared difference in value between s and its successor states:

$$\varphi_\pi(s) = \mathbb{E}_{s' \sim P_\pi(\cdot | s)} \left[(V_\pi(s) - V_\pi(s'))^2 \right]. \quad (15)$$

When the state space $\mathcal{S}$ is finite, VPS can be formulated as

$$\varphi_\pi(s) = \sum_{s' \in \mathcal{S}} P_\pi(s' | s) (V_\pi(s) - V_\pi(s'))^2 \quad (16)$$

where $P_\pi(s' | s)$ is the policy-induced transition probability, given for discrete action spaces by $P_\pi(s' | s) = \sum_{a \in \mathcal{A}} \pi(a | s) \mathcal{P}(s' | s, a)$.

For the reversible construction above, (9) and $W = D_d P_\pi$ give the exact relation $\mathcal{P}_i^{\text{elec}} = (d_i/2) \varphi_\pi(s_i)$. VPS is therefore degree-normalized node power: it measures the local value discrepancy per unit stationary mass rather than unnormalized electrical power. High-VPS states indicate bottlenecks of reward diffusion in the state space.

4.2. Online Estimate of VPS

Since the value of VPS depends on the value function V_π , we have proposed a simultaneous update law of V_π and φ_π as below.

$$\begin{cases} V_{t+1}(S_t) \leftarrow V_t(S_t) + \alpha_t [R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)] \\ \varphi_{t+1}(S_t) \leftarrow \varphi_t(S_t) + \beta_t [(V_t(S_t) - V_t(S_{t+1}))^2 - \varphi_t(S_t)] \end{cases} \quad (17)$$

The convergence of the update law is proved by Proposition 1 in Appendix A.1.

Although (17) offers an online algorithm for VPS estimation, its definition still depends on a specific policy π and reward r . To preliminarily validate the effectiveness of VPS for bottleneck identification, we consider a MiniGrid environment “Rooms” which contains different types of bottlenecks (Figure 2). The agent starts from random positions, each episode has a fixed length, and follows a random walk policy with four primitive actions (up, down, left and right). For rewards, we examine: (i) assigning a reward of 1 to a few designated states (see “Value Power Strength” in Figure 2); and (ii) sequentially assigning a reward of 1 to each grid and aggregating the resulting VPS distributions (see “Cumulative Value Power Strength”). As shown in Figure 2, placing a reward near a bottleneck effectively highlights it via VPS, while the cumulative VPS closely matches the patterns of current flow and shortest path betweenness centrality (Brandes & Fleischer, 2005; Borgatti & Everett, 2006).

4.3. VPS-Based Option Discovery

Based on the above, VPS can reveal reward diffusion bottlenecks in MDPs under random walk policies with suitable reward design. However, two challenges remain for option discovery: (i) how to design rewards to estimate VPS; (ii) how to design options with VPS.

For the first challenge, we propose an orthogonal random reward construction based on QR decomposition. For each batch, let $n = |\mathcal{S}|$, choose $1 \leq k \leq n$, sample a Gaussian reward matrix $Z \in \mathbb{R}^{n \times k}$ with i.i.d. entries, and orthogonalize its columns using a reduced QR decomposition:

$$Z_{si} \sim \mathcal{N}(0, 1), \quad Z = HC, \quad r^{(i)}(s) = H_{si}. \quad (18)$$

Here $H, Z \in \mathbb{R}^{n \times k}$, $C \in \mathbb{R}^{k \times k}$ is upper triangular, and $H^\top H = I_k$, so the resulting rewards are mutually orthogonal within the batch. The rationale for this design can be understood from the following spectral result.

Table 1. Analogy between Kirchhoff’s current law, Bellman equation, and Poisson’s equation.

No.	Kirchhoff’s Current Law	Bellman Equation	Poisson’s Equation
1	External current input I	Weighted reward input $D_d r_\pi$	External source term ρ
2	Node voltage U	State value V_π	Scalar potential Φ
3	Graph Laplacian L_c	$D_d \mathcal{T} = (1 - \gamma)D_d + \gamma L_c$	Operator $-\nabla^2$
4	Conductance matrix W	Weighted transition $W = D_d P_\pi$	—

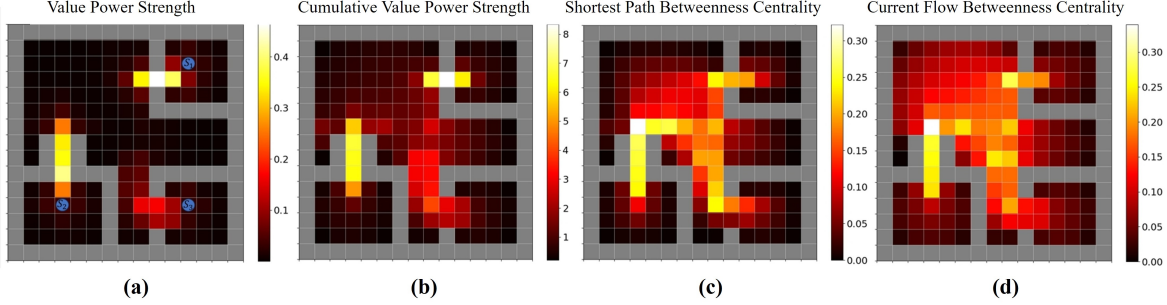


Figure 2. **VPS and betweenness centrality distribution in MiniGrid-Rooms:** (a) VPS distribution when the agent performs a random walk and only three selected grids provide a visit reward of 1 (all other grids give reward 0). (b) Cumulative VPS map obtained by sequentially assigning a visit reward of 1 to each grid (one target grid per experiment, all others 0), computing VPS for each case, and aggregating the resulting VPS fields. (c) Shortest-path betweenness centrality. (d) Current-flow betweenness centrality.

Lemma 1. Let $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ be a finite MDP with stationary policy π such that the induced Markov chain is reversible. Let $P_\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ denote the reversible policy-induced transition matrix, which is self-adjoint under the stationary-distribution weighted inner product, and define the random walk Laplacian as $L_{\text{rw}} = I - P_\pi$. The linear Bellman equation operator is denoted by $\mathcal{T} = I - \gamma P_\pi$ with $0 < \gamma < 1$. Then, for every eigenpair (μ_k, v_k) of L_{rw} , such that $L_{\text{rw}} v_k = \mu_k v_k$, the vector v_k is also an eigenvector of $\mathcal{T}$, with corresponding eigenvalue $\lambda_k = 1 - \gamma(1 - \mu_k)$.

Proof. See Appendix A.2. $\square$

Theorem 1 (Spectral Solution of the State-Value Function). Consider a finite, reversible MDP as in Lemma 1, and let P_π be the reversible policy-induced transition matrix with unique stationary distribution $\mathbf{d} \in \mathbb{R}^{|\mathcal{S}|}$ (i.e., $P_\pi^\top \mathbf{d} = \mathbf{d}$, $\sum_i d_i = 1$, $d_i > 0$). Under reversibility, P_π is self-adjoint under the $\mathbf{d}$ -weighted inner product. Let $D_d = \text{diag}(\mathbf{d})$ and $W = D_d P_\pi$; detailed balance makes W symmetric and its degree matrix is exactly D_d . Hence $L_{\text{rw}} = I - D_d^{-1} W = I - P_\pi$. Let $\{(\mu_k, v_k)\}_{k=1}^{|\mathcal{S}|}$ be the eigenpairs of L_{rw} with $\{v_k\}$ forming an orthonormal basis under the $\mathbf{d}$ -weighted inner product:

$$\langle f, g \rangle_{\mathbf{d}} := \sum_{i=1}^{|\mathcal{S}|} d_i f(s_i) g(s_i).$$

Let $\mathbf{r} \in \mathbb{R}^{|\mathcal{S}|}$ denote the reward vector whose i -th entry $\mathbf{r}_i$ is the expected immediate reward at s_i . Then, the solution V to the Bellman equation $\mathcal{T}V = \mathbf{r}$ admits the following

spectral decomposition:

$$V = \sum_{k=1}^{|\mathcal{S}|} \frac{1}{1 - \gamma(1 - \mu_k)} \langle \mathbf{r}, v_k \rangle_{\mathbf{d}} v_k. \quad (19)$$

Proof. See Appendix A.3. $\square$

The use of QR-orthogonalized random rewards for VPS is motivated by the spectral result of Theorem 1. In (18), we first sample a Gaussian reward matrix and then apply QR to obtain an orthonormal set of reward vectors (columns of H). This construction inherits the rotational invariance of Gaussian sampling and therefore has no preferred direction under the Euclidean inner product in expectation, while the orthogonality constraint reduces within-batch redundancy (Mezzadri, 2007). This Euclidean isotropy is distinct from isotropy under the $\mathbf{d}$ -weighted inner product used in Theorem 1. In expectation under the isotropic probes, the Bellman resolvent relatively amplifies components with smaller μ_k through the factor $[1 - \gamma(1 - \mu_k)]^{-1}$, favoring large-scale, smoothly varying structure (Chung, 1997).

The reversibility assumption is used only for this clean spectral interpretation of QR rewards; it is not required by the definition of VPS, the online estimator, or the option-learning pipeline. More generally, for any finite policy-induced chain, let $G = (I - \gamma P_\pi)^{-1}$ and let $g(s)$ be the row of G corresponding to state s . For Euclidean-isotropic QR reward probes h_i , where $\mathbb{E}_H$ denotes expectation over the random orthonormal reward matrix $H = [h_1, \dots, h_k]$,

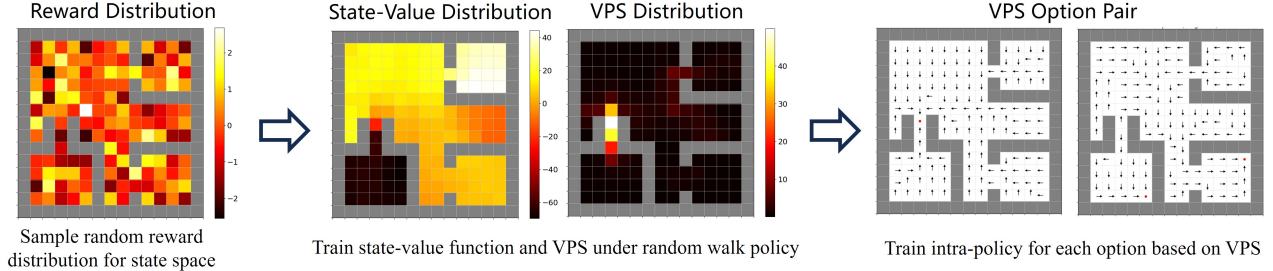


Figure 3. Pipeline for VPS-based option discovery

the expected averaged VPS satisfies

$$\mathbb{E}_H \left[\frac{1}{k} \sum_i \varphi_{h_i}(s) \right] = \frac{1}{|S|} \sum_{s'} P_\pi(s'|s) \|g(s) - g(s')\|_2^2. \quad (20)$$

Thus, in non-reversible MDPs, QR rewards still probe sharp one-step changes in discounted future reachability. High-VPS states therefore include classical undirected bottlenecks as well as directed transition interfaces, such as one-way transitions or irreversible object-state changes.

For the second challenge, inspired by the Eigenoption (Machado et al., 2018), the VPS-based dual potential-difference intrinsic rewards $r_{\text{int}}^{(m,\xi)}(s, s') = \xi (\varphi^{(m)}(s') - \varphi^{(m)}(s))$ with $\xi \in \{+1, -1\}$ are considered to obtain options towards or away from bottlenecks, where the superscript (m) denotes the m -th pair of options corresponding to the m -th random reward sampling.

The m -th pair of options is constructed from the intrinsic reward $r_{\text{int}}^{(m,\xi)}(s, s')$ under option discount γ_Q . To allow the option to stop whenever continuation has non-positive value, its optimal action-value function satisfies

$$Q_o^*(s, a) = \mathbb{E}[r_{\text{int}}^{(m,\xi)}(s, s') + \gamma_Q \max_{a'} \{0, \max_{a'} Q_o^*(s', a')\} \mid s, a]. \quad (21)$$

Here the zero continuation value represents terminating the option. The corresponding greedy intra-option policy is given by $a_o^*(s) = \arg \max_{a \in \mathcal{A}} Q_o^*(s, a)$.

The initiation set of each option is set as the entire state space $\mathcal{S}$. The termination probability $\beta_o(s)$ is defined as

$$\beta_o(s) = \begin{cases} 1, & \text{if } \max_a Q_o^*(s, a) \leq 0 \\ 1/N, & \text{otherwise} \end{cases} \quad (22)$$

where N is a manually specified positive constant controlling stochastic termination during option construction. It is distinct from the experimental execution horizon L , which is a hard cap on an option rollout. The options designed in this manner enable the agent to reach states with the highest

or lowest VPS values (or local extrema, depending on the discount factor).

Figure 3 illustrates the overall procedure of VPS-based option discovery, while the algorithmic procedure is provided in Appendix B.

4.4. Function Approximation Cases

To generalize VPS to high-dimensional or continuous state spaces, we learn a shared encoder $z_\omega(s) \in \mathbb{R}^d$ with a value head V_θ and a VPS head φ_η . We train V_θ with a target encoder $z_{\bar{\omega}}$ and target value head $V_{\bar{\theta}}$ and estimate VPS as a conditional regression target built from $V_{\bar{\theta}}$:

$$\mathcal{L}_V(\theta, \omega) = \mathbb{E}_{(s, r, s') \sim \mathcal{D}} [r + \gamma V_{\bar{\theta}}(z_{\bar{\omega}}(s')) - V_\theta(z_\omega(s))]^2, \quad (23)$$

$$\mathcal{L}_\varphi(\eta, \omega) = \mathbb{E}_{(s, s') \sim \mathcal{D}} [(V_{\bar{\theta}}(z_{\bar{\omega}}(s)) - V_{\bar{\theta}}(z_{\bar{\omega}}(s')))^2 - \varphi_\eta(z_\omega(s))]^2. \quad (24)$$

In practice, we optionally stop the gradient from $\mathcal{L}_\varphi$ to z_ω if training is unstable.

For high-dimensional or continuous spaces, we construct RFF rewards (Rahimi & Recht, 2007) on $z_\omega(s)$ (see Preliminaries and (5)). The resulting fields are continuous in z_ω ; when the representation aligns with transition dynamics, this provides a practical smoothness proxy that stabilizes VPS estimation.

5. Experimental Results

We conduct experiments on tabular MiniGrid-KeyLock and Taxi-v3, continuous-control PointMaze-FourRooms, and Atari 2600 environments. The tabular and PointMaze experiments collect transitions under random-walk policies, whereas Atari uses an online-trained Random Network Distillation (RND) exploration policy (Burda et al., 2019). VPS is trained with random rewards from (18) or (5), as appropriate.

5.1. Tabular Cases

For the tabular cases, we evaluate on MiniGrid-KeyLock (Chevalier-Boisvert et al., 2023) and Gym Taxi-v3 (Brockman et al., 2016). We compare four settings: **Primi-**

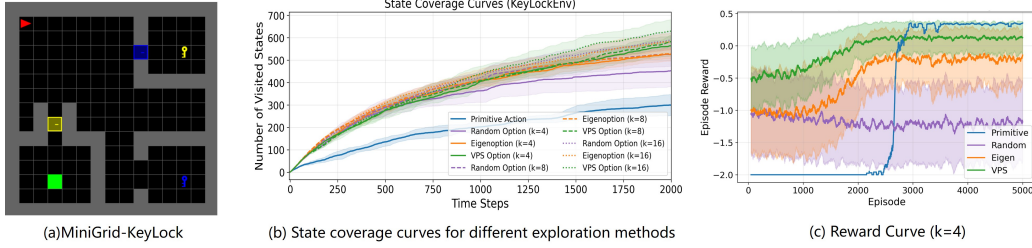


Figure 4. (a) MiniGrid-KeyLock environment. (b) State coverage under uniform selection over primitive actions and startable options, averaged over 5 seeds. (c) Q-learning with option exploration ($k=4$ option pairs).

tive (no options; only primitive actions), **Random Option** (assign each state a Gaussian random potential and train an option pair to reach the max/min-potential states), **Eigenoption** (implemented following Machado et al. (2017)), and **VPS Option** (our method).

MiniGrid-KeyLock. To evaluate whether VPS options improve exploration, we first consider a gridworld with a blue/yellow key, matching doors, and a goal; the agent starts at (1, 1). Observations provide global object-state values (agent position/orientation, door open/closed, key on-map vs. carried/consumed). The action space has 6 actions (up/down/left/right, pick up, toggle door). Reward is +1 at the goal (terminate), otherwise -0.01 per step. Detailed experimental settings are provided in Appendix C.1.

Figure 5 visualizes phase-conditioned VPS heatmaps for a representative option in MiniGrid-KeyLock. High-VPS regions concentrate around doors, keys, and key-dependent transition states, showing that VPS captures task-semantic bottlenecks beyond geometric passages.

We report two main MiniGrid-KeyLock results: (i) state coverage under option-based random walk (Figure 4(b)); and (ii) Q-learning with option exploration (Figure 4(c)). VPS consistently improves coverage and learning speed. Additional goal-reaching success distributions under random walk are reported in Appendix C.2, with the detailed settings in Appendix C.1. The **Primitive** return may become higher later since we plot training returns (no separate evaluation), and executing options can increase steps-to-goal.

Taxi-v3. We evaluate temporal credit assignment in Taxi-v3 as an SMDP. During reward collection, the agent follows an ϵ -greedy policy over primitive actions and learned options; exploratory choices are sampled uniformly from this augmented action set, and option returns are credited by SMDP Q-learning. Each method yields 20 options: VPS uses 10 intrinsic rewards (two directions each), eigenoptions use the 10 Laplacian eigenvectors associated with the smallest eigenvalues, and the random baseline uses 20 random options. We generate 10 option sets per method with different seeds and evaluate each set over 10 training runs. Figure 6(a) shows a representative VPS option that completes pickup

and drop-off without external-reward training by reaching task-critical bottleneck configurations. Figures 6(b) and 6(c) report random-walk task completions and Q-learning returns, respectively. VPS options yield meaningful behaviors and accelerate reward acquisition, although they do not always improve final performance.

5.2. Continuous-Control PointMaze

To evaluate whether VPS options remain useful beyond discrete state-action spaces, we consider a continuous-control PointMaze-FourRooms task, where narrow passages form natural bottlenecks. During reward-free pretraining, VPS first collects 500,000 random-walk transitions to fit RFF value/VPS estimators and then trains eight SAC option policies, while DIAYN trains eight reward-free skills. Figure 7 shows that RFF-induced value functions are smooth over the maze, whereas VPS concentrates near passage-like bottlenecks and yields positive/negative options with complementary rollouts.

For downstream evaluation, we compare SAC (Haarnoja et al., 2018), SAC with DIAYN-assisted exploration (Eysenbach et al., 2019), and SAC with VPS-assisted exploration. In the sequential waypoint task (Figure 8(a)), episodes start from (2, 2), last at most 1000 steps, and give reward +1 only when four waypoints are visited in order. Detailed downstream and intervention settings are provided in Appendix C.1, Table 11. In this experiment, VPS options and DIAYN skills are used only as exploration tools and do not participate in credit assignment; the RL process remains a standard MDP rather than an SMDP, and SAC updates still optimize the external waypoint reward. Figure 8(b) shows that VPS options achieve higher mean evaluation reward, indicating that bottleneck-oriented skills provide useful temporal abstraction. Representative DIAYN rollouts are shown in Appendix Figure 17.

5.3. Atari 2600

To verify whether VPS can identify bottlenecks in high-dimensional visual observations, we consider Atari 2600 games. An online-trained RND exploration policy (Burda

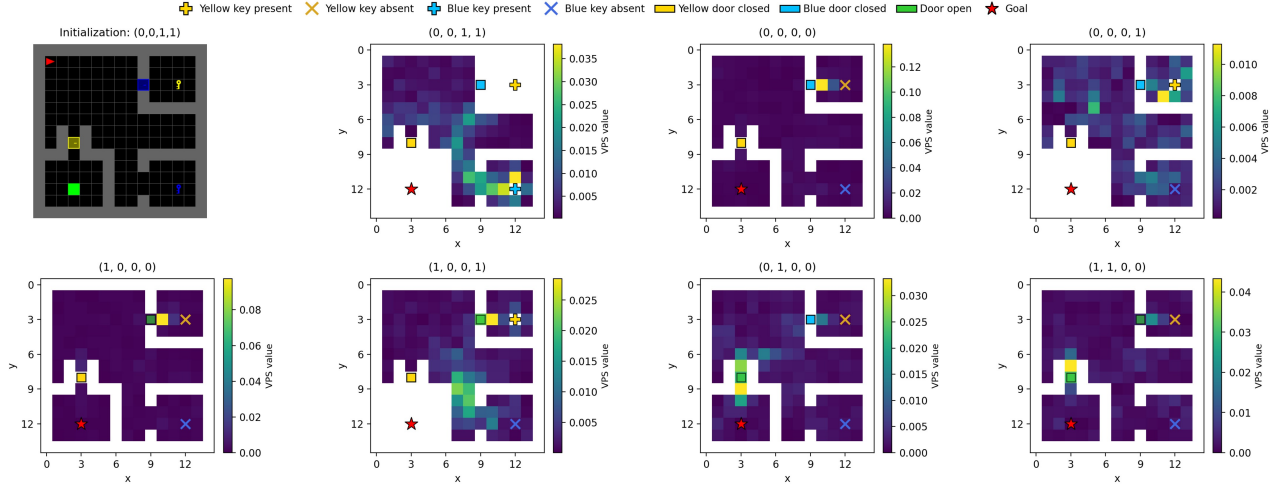


Figure 5. VPS heatmaps for a representative option in MiniGrid-KeyLock. The top-left panel shows the initialization rendering. The remaining seven panels are phase-conditioned heatmaps titled by their (a, b, c, d) configuration, where $(a, b, c, d) = (\text{blue_door_open}, \text{yellow_door_open}, \text{blue_key_on_map}, \text{yellow_key_on_map})$. Each heatmap value at (x, y) is obtained by averaging the values of all reachable $(x, y, \text{direction})$ states and projecting that mean onto position (x, y) ; brighter regions indicate stronger reward-diffusion bottlenecks.

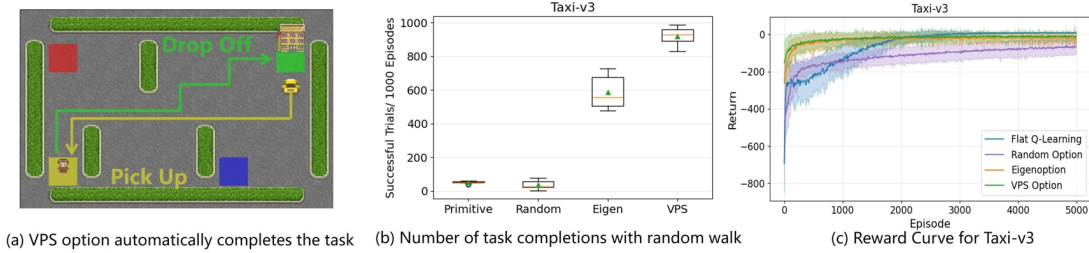


Figure 6. Performance of different options in Taxi-v3. A “successful trial” refers to an episode in which the taxi successfully transports a passenger from the starting point to the destination once.

et al., 2019) collects transition data. Using single-frame 84×84 grayscale images as input, we train eight VPS estimators from eight independent RFF reward probes. We then evaluate human-gameplay videos in which a player guides the agent to explore the state space. For each frame, we aggregate the VPS outputs of the eight trained estimators; no option policy is trained or executed in the Atari experiments. Figure 9 illustrates that in the ALE/Venture-v5 environment, peaks of the aggregated VPS score reliably correspond to interpretable bottleneck frames—specifically, doorways between different rooms (the positions of the agent are marked by white cross stars). Additional results for more Atari games are provided in Appendix C.2.

6. Limitations and Future Work

A practical limitation of VPS is computational cost: estimating bottlenecks from many random reward functions may require many corresponding value predictions, and a direct implementation scales linearly with the number of reward probes. A natural remedy is to share computation across

probes by learning a common transition-aware encoder and temporal-difference targets, while using lightweight reward-specific heads to predict each induced value field and VPS target. This preserves reward diversity while avoiding repeated learning of the same transition structure.

Successor representations provide a principled route for such sharing, since they separate environment dynamics from rewards and allow values to be recomputed when reward weights change (Dayan, 1993; Barreto et al., 2017; 2018). Future work could combine VPS with successor-feature modules, deep successor representation learning (Kulkarni et al., 2016), or Proto-Value Networks (Farebrother et al., 2023) to compute many reward-conditioned value functions and VPS maps in parallel.

7. Conclusion

This paper introduces Value Power Strength (VPS), a value function-based metric for identifying reward-diffusion bottlenecks without constructing an explicit transition graph.

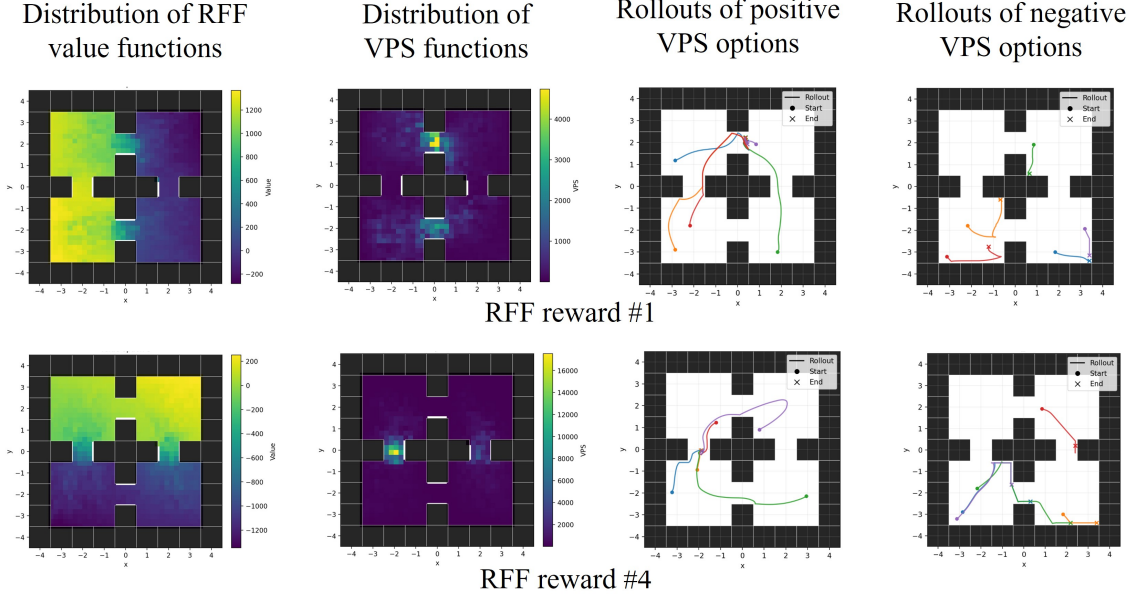


Figure 7. VPS estimation and option rollouts in continuous-control PointMaze-FourRooms. For two RFF reward instances, the learned value functions diffuse smoothly through the maze, while VPS concentrates near narrow passages that mediate reward propagation. Positive and negative VPS options induce complementary rollouts toward different extrema of the VPS field.

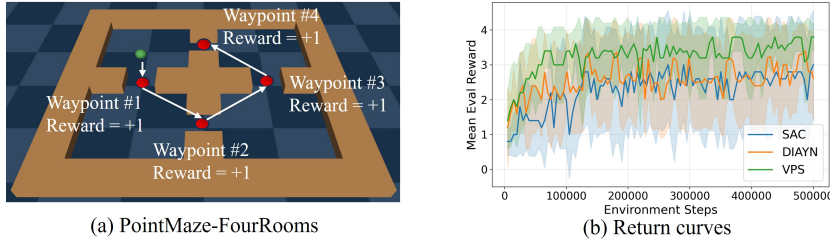


Figure 8. Downstream evaluation in PointMaze-FourRooms. (a) Environment and sequential waypoint task: the agent starts from (2, 2) and receives reward +1 only when visiting the four waypoints in order. (b) Mean evaluation reward over downstream environment steps for SAC, SAC with DIAYN-assisted exploration, and SAC with VPS-assisted exploration. Pretrained DIAYN skills and VPS options are invoked only as exploration interventions during SAC training.

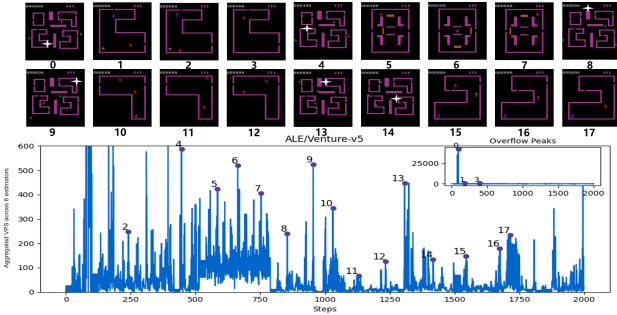


Figure 9. Bottlenecks identified by the VPS score aggregated across eight independently trained estimators in ALE/Venture-v5.

Motivated by the analogy between node-wise power dissipation and squared value discrepancies, VPS provides a local signal that can be estimated in tabular domains and with function approximation. We further use VPS as a potential to learn interpretable option pairs that steer toward or away from high-VPS regions.

The resulting framework links bottleneck discovery, value estimation, and option learning within a unified pipeline that can extend beyond explicit graph representations. Empirically, VPS highlights task-relevant transition regions such as doors, key-dependent states, narrow passages, and salient Atari frames, suggesting that reward-diffusion structure is a useful principle for interpretable temporal abstraction in sparse-reward domains.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant Nos. 62236007, 32441113, and 62276235. The authors sincerely thank the anonymous reviewers and chairs for their constructive comments, which helped improve the quality of this work.

Impact Statement

This work aims to improve interpretable and sample-efficient hierarchical reinforcement learning by identifying reward-diffusion bottlenecks. Its potential risks are those common to reinforcement learning methods: better exploration and temporal abstraction should be deployed with task-specific safety constraints in sensitive or real-world settings.

References

- Bagaria, A., Abbatematteo, B., Gottesman, O., Corsaro, M., Rammohan, S., and Konidaris, G. Effectively learning initiation sets in hierarchical reinforcement learning. *Advances in Neural Information Processing Systems*, 36: 73441–73463, 2023.
- Barreto, A., Dabney, W., Munos, R., Hunt, J. J., Schaul, T., van Hasselt, H. P., and Silver, D. Successor features for transfer in reinforcement learning. *Advances in Neural Information Processing Systems*, 30, 2017.
- Barreto, A., Borsa, D., Quan, J., Schaul, T., Silver, D., Hessel, M., Mankowitz, D., Zidek, A., and Munos, R. Transfer in deep reinforcement learning using successor features and generalised policy improvement. In *International Conference on Machine Learning*, pp. 501–510. PMLR, 2018.
- Belkin, M. and Niyogi, P. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003.
- Borgatti, S. P. and Everett, M. G. A graph-theoretic perspective on centrality. *Social Networks*, 28(4):466–484, 2006.
- Brandes, U. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.
- Brandes, U. and Fleischer, D. Centrality measures based on current flow. In *Annual Symposium on Theoretical Aspects of Computer Science*, pp. 533–544. Springer, 2005.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by Random Network Distillation. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=H11JJnR5Ym>.
- Chandra, A. K., Raghavan, P., Ruzzo, W. L., and Smolensky, R. The electrical resistance of a graph captures its commute and cover times. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, pp. 574–586, 1989.
- Chevalier-Boisvert, M., Dai, B., Towers, M., Perez-Vicente, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. MiniGrid & MiniWorld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems*, volume 36, pp. 73383–73394, December 2023.
- Chung, F. R. K. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997.
- Dayan, P. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993.
- Doyle, P. G. and Snell, J. L. *Random walks and electric networks*, volume 22. American Mathematical Soc., 1984.
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. First return, then explore. *Nature*, 590(7847): 580–586, 2021.
- Eysenbach, B., Gupta, A., Ibarz, J., and Levine, S. Diversity is all you need: Learning skills without a reward function. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SJx63jRqFm>.
- Farebrother, J., Greaves, J., Agarwal, R., Le Lan, C., Goroshin, R., Castro, P. S., and Bellemare, M. G. Proto-value networks: Scaling representation learning with auxiliary tasks. In *International Conference on Learning Representations*, 2023.
- Freeman, L. C. A set of measures of centrality based on betweenness. *Sociometry*, pp. 35–41, 1977.
- Gomez, D., Bowling, M., and Machado, M. C. Proper Laplacian representation learning. In *International Conference on Learning Representations*, 2024.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Ma-*

- chine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- Ivanov, A., Bagaria, A., and Konidaris, G. Discovering options that minimize average planning time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 17573–17581, 2025.
- Jinnai, Y., Abel, D., Hershkowitz, D., Littman, M., and Konidaris, G. Finding options that minimize planning time. In *International Conference on Machine Learning*, pp. 3120–3129. PMLR, 2019a.
- Jinnai, Y., Park, J. W., Abel, D., and Konidaris, G. Discovering options for exploration by minimizing cover time. In *International Conference on Machine Learning*, pp. 3130–3139. PMLR, 2019b.
- Klissarov, M. and Machado, M. C. Deep Laplacian-based options for temporally-extended exploration. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- Kotamreddy, H. and Machado, M. C. A study of value-aware eigenoptions. In *IBRL @ RLC*, 2025. URL <https://openreview.net/forum?id=ZHlMV2B49W>.
- Kulkarni, T. D., Saeedi, A., Gautam, S., and Gershman, S. J. Deep successor reinforcement learning. *arXiv preprint arXiv:1606.02396*, 2016.
- Lyle, C., Rowland, M., Ostrovski, G., and Dabney, W. On the effect of auxiliary tasks on representation dynamics. In *International Conference on Artificial Intelligence and Statistics*, pp. 1–9. PMLR, 2021.
- Machado, M. C., Bellemare, M. G., and Bowling, M. A Laplacian framework for option discovery in reinforcement learning. In *International Conference on Machine Learning*, pp. 2295–2304. PMLR, 2017.
- Machado, M. C., Rosenbaum, C., Guo, X., Liu, M., Tesauro, G., and Campbell, M. Eigenoption discovery through the deep successor representation. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=Bk8ZcAxR->.
- Machado, M. C., Barreto, A., Precup, D., and Bowling, M. Temporal abstraction in reinforcement learning with the successor representation. *Journal of Machine Learning Research*, 24(80):1–69, 2023.
- Mahadevan, S. Proto-value functions: Developmental reinforcement learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pp. 553–560, 2005.
- Mahadevan, S. and Maggioni, M. Proto-value functions: A Laplacian framework for learning representation and control in Markov Decision Processes. *Journal of Machine Learning Research*, 8(10), 2007.
- Menache, I., Mannor, S., and Shimkin, N. Q-cut: Dynamic discovery of subgoals in reinforcement learning. In *Machine Learning: ECML 2002: 13th European Conference on Machine Learning Helsinki, Finland, August 19–23, 2002 Proceedings 13*, pp. 295–306. Springer, 2002.
- Mezzadri, F. How to generate random matrices from the classical compact groups. *Notices of the American Mathematical Society*, 54(5):592–604, May 2007. URL <https://www.ams.org/notices/200705/fea-mezzadri-web.pdf>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Hiedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- Nachum, O., Gu, S. S., Lee, H., and Levine, S. Data-efficient hierarchical reinforcement learning. *Advances in Neural Information Processing Systems*, 31, 2018.
- Nachum, O., Tang, H., Lu, X., Gu, S., Lee, H., and Levine, S. Why does hierarchy (sometimes) work so well in reinforcement learning? *arXiv preprint arXiv:1909.10618*, 2019.
- Newman, M. E. A measure of betweenness centrality based on random walks. *Social Networks*, 27(1):39–54, 2005.
- Park, S., Ghosh, D., Eysenbach, B., and Levine, S. HIQL: Offline goal-conditioned RL with latent states as actions. *Advances in Neural Information Processing Systems*, 36: 34866–34891, 2023.
- Pateria, S., Subagdja, B., Tan, A.-h., and Quek, C. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 54(5):1–35, 2021.
- Puterman, M. L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., 1994.
- Rahimi, A. and Recht, B. Random features for large-scale kernel machines. *Advances in Neural Information Processing Systems*, 20, 2007.

- Ramesh, A., Kirsch, L., van Steenkiste, S., and Schmidhuber, J. Exploring through random curiosity with general value functions. *Advances in Neural Information Processing Systems*, 35:18733–18748, 2022.
- Ryu, J. J., Zhou, S., and Wornell, G. W. Revisiting orbital minimization method for neural operator decomposition. In *Advances in Neural Information Processing Systems*, 2025.
- Shi, J. and Malik, J. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.
- Şimşek, Ö. and Barto, A. Skill characterization based on betweenness. *Advances in Neural Information Processing Systems*, 21, 2008.
- Şimşek, Ö., Wolfe, A. P., and Barto, A. G. Identifying useful subgoals in reinforcement learning by local graph partitioning. In *Proceedings of the 22nd International Conference on Machine Learning*, pp. 816–823, 2005.
- Solway, A., Diuk, C., Córdova, N., Yee, D., Barto, A. G., Niv, Y., and Botvinick, M. M. Optimal behavioral hierarchy. *PLOS Computational Biology*, 10(8):e1003779, 2014.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- Sutton, R. S., Precup, D., and Singh, S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2): 181–211, 1999.
- Tetali, P. Random walks and the effective resistance of networks. *Journal of Theoretical Probability*, 4(1):101–109, 1991.
- Touati, A., Rapin, J., and Ollivier, Y. Does zero-shot reinforcement learning exist? In *International Conference on Learning Representations*, 2023.
- Tsitsiklis, J. and Van Roy, B. Analysis of temporal-difference learning with function approximation. *Advances in Neural Information Processing Systems*, 9, 1996.
- von Luxburg, U. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.
- Wang, K., Zhou, K., Zhang, Q., Shao, J., Hooi, B., and Feng, J. Towards better Laplacian representation in reinforcement learning with generalized graph drawing. In *International Conference on Machine Learning*, pp. 11003–11012. PMLR, 2021.
- Wu, Y., Tucker, G., and Nachum, O. The Laplacian in RL: Learning representations with efficient approximations. In *International Conference on Learning Representations*, 2019.
- Zheng, Z., Veeriah, V., Vuorio, R., Lewis, R. L., and Singh, S. Learning state representations from random deep action-conditional predictions. *Advances in Neural Information Processing Systems*, 34:23679–23691, 2021.

Appendix Contents

A	Proofs of Main Results	14
A.1	Proposition 1 and its proof	14
A.2	Proof of Lemma 1	16
A.3	Proof of Theorem 1	17
B	Implementation of VPS-Based Option Discovery	19
C	Experimental Details and Additional Results	20
C.1	Detailed Experimental Settings	20
C.2	Supplementary Experimental Results	25

A. Proofs of Main Results

A.1. Proposition 1 and its proof

$$\begin{cases} V_{t+1}(S_t) \leftarrow V_t(S_t) + \alpha_t [R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)] \\ \varphi_{t+1}(S_t) \leftarrow \varphi_t(S_t) + \beta_t [(V_t(S_t) - V_t(S_{t+1}))^2 - \varphi_t(S_t)] \end{cases} \quad (25)$$

Proposition 1. Consider a finite MDP defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ with a stationary policy π . Let $\{V_t\}_{t \geq 0}$ and $\{\varphi_t\}_{t \geq 0}$ be the sequences of value estimates and VPS estimates updated according to (25). Suppose the following conditions hold:

1. The Markov chain induced by π is ergodic (irreducible and aperiodic);
2. The learning rates $\{\alpha_t\}$ and $\{\beta_t\}$ are positive, deterministic, and at most one. For every $s \in \mathcal{S}$, if $\{\tau_k(s)\}_{k \geq 1}$ are the times at which $S_t = s$, their visit subsequences satisfy the state-wise Robbins–Monro conditions:

$$\sum_k \alpha_{\tau_k(s)} = \sum_k \beta_{\tau_k(s)} = \infty, \quad \sum_k \alpha_{\tau_k(s)}^2 < \infty, \quad \sum_k \beta_{\tau_k(s)}^2 < \infty;$$

3. The second moment of the reward is uniformly bounded:

$$\mathbb{E}_\pi [R_{t+1}^2 \mid S_t = s] \leq R_{\max}^2 < \infty, \quad \forall s \in \mathcal{S}.$$

Then, for $\forall s \in \mathcal{S}$, the sequences $\{V_t\}$ and $\{\varphi_t\}$ converge almost surely to the true state-value function V_π and the true VPS function φ_π , respectively.

Proof. The convergence of the value function sequence $\{V_t\}$ to V_π under the given conditions is a well-established result (Tsitsiklis & Van Roy, 1996). We thus focus on proving the almost sure convergence of the VPS sequence $\{\varphi_t\}$ to φ_π .

Step 1: Preliminaries and Notation

Since the Markov chain induced by π is ergodic, every state $s \in \mathcal{S}$ is visited infinitely often almost surely. Let $\{\tau_k(s)\}_{k=1}^\infty$ denote the increasing sequence of time steps at which state s is visited. Consider the subsequence $\{\varphi_{\tau_k(s)}\}_{k \geq 1}$.

It is standard that, under the state-wise Robbins–Monro conditions and the assumptions on rewards, $V_t(s) \rightarrow V_\pi(s)$ almost surely for all $s \in \mathcal{S}$. Since $\mathcal{S}$ is finite, this convergence is uniform:

$$\forall \varepsilon > 0, \exists K(\varepsilon) \text{ such that } \forall k \geq K(\varepsilon), \max_{s \in \mathcal{S}} |V_{\tau_k}(s) - V_\pi(s)| < \varepsilon \quad \text{a.s.} \quad (26)$$

Step 2: Stochastic Update for VPS

By definition, when s is visited at time τ_k , the update is:

$$\begin{aligned} \varphi_{\tau_k+1}(s) &= \varphi_{\tau_k}(s) + \beta_{\tau_k} [(V_{\tau_k}(s) - V_{\tau_k}(S_{\tau_k+1}))^2 - \varphi_{\tau_k}(s)] \\ &= (1 - \beta_{\tau_k})\varphi_{\tau_k}(s) + \beta_{\tau_k} (V_{\tau_k}(s) - V_{\tau_k}(S_{\tau_k+1}))^2. \end{aligned}$$

Let $e_k(s) := \varphi_{\tau_k}(s) - \varphi_\pi(s)$ denote the error at the k -th visit to s . We have

$$\begin{aligned} e_{k+1}(s) &= (1 - \beta_{\tau_k})e_k(s) + \beta_{\tau_k} [(V_{\tau_k}(s) - V_{\tau_k}(S_{\tau_k+1}))^2 - \varphi_\pi(s)] \\ &= (1 - \beta_{\tau_k})e_k(s) + \beta_{\tau_k} (A_k + B_k) \end{aligned} \quad (27)$$

where we decompose

$$\begin{aligned} A_k &:= (V_{\tau_k}(s) - V_{\tau_k}(S_{\tau_k+1}))^2 - (V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))^2, \\ B_k &:= (V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))^2 - \mathbb{E}_{s' \sim P_{\pi}(\cdot|s)}[(V_{\pi}(s) - V_{\pi}(s'))^2]. \end{aligned}$$

Step 3: Uniform Control of the Bias Term A_k

Let $\delta_1 = V_{\tau_k}(s) - V_{\pi}(s)$, $\delta_2 = V_{\tau_k}(S_{\tau_k+1}) - V_{\pi}(S_{\tau_k+1})$. Then,

$$\begin{aligned} A_k &= (V_{\tau_k}(s) - V_{\tau_k}(S_{\tau_k+1}))^2 - (V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))^2 \\ &= ((V_{\pi}(s) + \delta_1) - (V_{\pi}(S_{\tau_k+1}) + \delta_2))^2 - (V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))^2 \\ &= ((V_{\pi}(s) - V_{\pi}(S_{\tau_k+1})) + (\delta_1 - \delta_2))^2 - (V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))^2 \\ &= 2(V_{\pi}(s) - V_{\pi}(S_{\tau_k+1}))(\delta_1 - \delta_2) + (\delta_1 - \delta_2)^2. \end{aligned}$$

Applying triangle inequality and uniform convergence (26), when k is large enough,

$$|\delta_1|, |\delta_2| < \varepsilon, \quad |V_{\pi}(s) - V_{\pi}(S_{\tau_k+1})| \leq 2M,$$

where $M = \max_{s \in \mathcal{S}} |V_{\pi}(s)|$. Thus,

$$|A_k| \leq 2 \cdot 2M \cdot 2\varepsilon + (2\varepsilon)^2 = 8M\varepsilon + 4\varepsilon^2.$$

Therefore, $A_k \rightarrow 0$ almost surely as $k \rightarrow \infty$.

Step 4: Martingale Difference Property of B_k

For each k , B_k is a martingale difference: conditioned on $\mathcal{F}_{\tau_k}$ (the σ -algebra up to time τ_k),

$$\mathbb{E}[B_k | \mathcal{F}_{\tau_k}] = 0.$$

Moreover, since $|V_{\pi}(s)| \leq M$, it follows $|B_k| \leq (2M)^2 = 4M^2$ almost surely, so the sequence has uniformly bounded second moment.

Step 5: Convergence via Auxiliary Sequence Construction

The error dynamics in (27) can be expressed as:

$$e_{k+1}(s) = (1 - \beta_{\tau_k})e_k(s) + \beta_{\tau_k}w_k + \beta_{\tau_k}\delta_k \quad (28)$$

where:

$$\begin{aligned} w_k &:= B_k + (A_k - \mathbb{E}[A_k | \mathcal{F}_{\tau_k}]) \\ \delta_k &:= \mathbb{E}[A_k | \mathcal{F}_{\tau_k}] \end{aligned}$$

By construction, w_k satisfies:

- (a) $\mathbb{E}[w_k | \mathcal{F}_{\tau_k}] = 0$ (martingale difference)
- (b) $|w_k| \leq |B_k| + |A_k| + \mathbb{E}[|A_k| | \mathcal{F}_{\tau_k}] \leq 4M^2 + 16M\varepsilon + 8\varepsilon^2 < \infty$ a.s.

Thus $\mathbb{E}[w_k^2 | \mathcal{F}_{\tau_k}] \leq C$ for some $C < \infty$ almost surely.

Consider the auxiliary sequence defined by:

$$\tilde{e}_{k+1}(s) = (1 - \beta_{\tau_k})\tilde{e}_k(s) + \beta_{\tau_k}w_k, \quad \tilde{e}_0(s) = e_0(s) \quad (29)$$

To apply Lemma 1 in (Tsitsiklis & Van Roy, 1996), the following conditions should be satisfied:

- (a) $\mathbb{E}[w_k \mid \mathcal{F}_{\tau_k}] = 0$
- (b) $\mathbb{E}[w_k^2 \mid \mathcal{F}_{\tau_k}] \leq C < \infty$
- (c) $\beta_{\tau_k} \in (0, 1]$ (by assumption)
- (d) $\sum_{k=0}^{\infty} \beta_{\tau_k} = \infty$ (state-wise Robbins–Monro assumption)
- (e) $\sum_{k=0}^{\infty} \beta_{\tau_k}^2 < \infty$ (state-wise Robbins–Monro assumption)

Under these conditions, Lemma 1 implies $\tilde{e}_k(s) \rightarrow 0$ almost surely as $k \rightarrow \infty$.

The difference between the sequences satisfies:

$$|e_k(s) - \tilde{e}_k(s)| \leq \sum_{m=0}^{k-1} \beta_{\tau_m} |\delta_m| \prod_{j=m+1}^{k-1} (1 - \beta_{\tau_j}).$$

Fix any $\eta > 0$. Since $\delta_m \rightarrow 0$ almost surely (from Step 3), there exists K such that $|\delta_m| \leq \eta$ for all $m \geq K$ on the almost sure event of convergence. Splitting the sum at K gives

$$\begin{aligned} |e_k(s) - \tilde{e}_k(s)| &\leq \sum_{m=0}^{K-1} \beta_{\tau_m} |\delta_m| \prod_{j=m+1}^{k-1} (1 - \beta_{\tau_j}) \\ &\quad + \eta \sum_{m=K}^{k-1} \beta_{\tau_m} \prod_{j=m+1}^{k-1} (1 - \beta_{\tau_j}). \end{aligned}$$

For each fixed $m < K$, the product term converges to zero because $\sum_j \beta_{\tau_j} = \infty$ and $\beta_{\tau_j} \in (0, 1]$. Hence the finite early sum vanishes as $k \rightarrow \infty$. The tail sum is bounded by

$$\sum_{m=K}^{k-1} \beta_{\tau_m} \prod_{j=m+1}^{k-1} (1 - \beta_{\tau_j}) = 1 - \prod_{j=K}^{k-1} (1 - \beta_{\tau_j}) \leq 1,$$

so $\limsup_{k \rightarrow \infty} |e_k(s) - \tilde{e}_k(s)| \leq \eta$. Since η is arbitrary, $|e_k(s) - \tilde{e}_k(s)| \rightarrow 0$ almost surely.

Combining with $\tilde{e}_k(s) \rightarrow 0$ a.s., we conclude $e_k(s) \rightarrow 0$ almost surely.

Step 6: Extension to the Full Sequence

The above argument holds for each $s \in \mathcal{S}$. Since the state space is finite and $\varphi_t(s)$ is only updated at visits to s , the entire sequence $\{\varphi_t(s)\}_{t \geq 0}$ converges almost surely to $\varphi_\pi(s)$. This completes the proof. $\square$

A.2. Proof of Lemma 1

Lemma 1. *Let $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ be a finite MDP with stationary policy π such that the induced Markov chain is reversible. Let $P_\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ denote the reversible policy-induced transition matrix, which is self-adjoint under the stationary-distribution weighted inner product, and define the random walk Laplacian as $L_{\text{rw}} = I - P_\pi$. The linear Bellman equation operator is denoted by $\mathcal{T} = I - \gamma P_\pi$ with $0 < \gamma < 1$. Then, for every eigenpair (μ_k, v_k) of L_{rw} , such that $L_{\text{rw}} v_k = \mu_k v_k$, the vector v_k is also an eigenvector of $\mathcal{T}$, with corresponding eigenvalue $\lambda_k = 1 - \gamma(1 - \mu_k)$.*

Proof. Let (μ_k, v_k) be an eigenpair of the random walk Laplacian L_{rw} , satisfying:

$$L_{\text{rw}} v_k = \mu_k v_k$$

By definition $L_{\text{rw}} = I - P_\pi$, we have:

$$(I - P_\pi) v_k = \mu_k v_k \tag{30}$$

Rearranging terms yields:

$$P_\pi v_k = (1 - \mu_k)v_k \quad (31)$$

Now consider the linear Bellman equation operator $\mathcal{T} = I - \gamma P_\pi$:

$$\mathcal{T}v_k = (I - \gamma P_\pi)v_k = v_k - \gamma P_\pi v_k$$

Substituting (31):

$$\mathcal{T}v_k = v_k - \gamma(1 - \mu_k)v_k = [1 - \gamma(1 - \mu_k)]v_k$$

Thus, v_k is an eigenvector of $\mathcal{T}$ with eigenvalue $\lambda_k = 1 - \gamma(1 - \mu_k)$. $\square$

A.3. Proof of Theorem 1

Theorem 1 (Spectral Solution of the State-Value Function). *Consider a finite, reversible MDP as in Lemma 1, and let P_π be the policy-induced transition matrix with unique stationary distribution $\mathbf{d} \in \mathbb{R}^{|S|}$ (i.e., $P_\pi^\top \mathbf{d} = \mathbf{d}$, $\sum_i d_i = 1$, $d_i > 0$). Let $D_d = \text{diag}(\mathbf{d})$ and $W = D_d P_\pi$; detailed balance makes W symmetric and its degree matrix is D_d . Hence $L_{\text{rw}} = I - D_d^{-1}W = I - P_\pi$. Let $\{(\mu_k, v_k)\}_{k=1}^{|S|}$ be the eigenpairs of L_{rw} with $\{v_k\}$ forming an orthonormal basis under the $\mathbf{d}$ -weighted inner product:*

$$\langle f, g \rangle_{\mathbf{d}} := \sum_{i=1}^{|S|} d_i f(s_i) g(s_i).$$

Let $\mathbf{r} \in \mathbb{R}^{|S|}$ denote the reward vector whose i -th entry $\mathbf{r}_i$ is the expected immediate reward at s_i . Then, the solution V to the Bellman equation $\mathcal{T}V = \mathbf{r}$ admits the following spectral decomposition:

$$V = \sum_{k=1}^{|S|} \frac{1}{1 - \gamma(1 - \mu_k)} \langle \mathbf{r}, v_k \rangle_{\mathbf{d}} v_k. \quad (32)$$

Proof. The Bellman equation for the state-value function is given by:

$$\mathcal{T}V = \mathbf{r}$$

where $\mathcal{T} = I - \gamma P_\pi$ is the linear Bellman equation operator. This can be rewritten as:

$$(I - \gamma P_\pi)V = \mathbf{r} \quad (33)$$

By Lemma 1, for each eigenpair (μ_k, v_k) of $L_{\text{rw}} = I - P_\pi$, v_k is also an eigenvector of $\mathcal{T}$ with eigenvalue:

$$\lambda_k = 1 - \gamma(1 - \mu_k)$$

Since P_π is reversible, it is self-adjoint under the $\mathbf{d}$ -weighted inner product. Therefore, $L_{\text{rw}} = I - P_\pi$ and $\mathcal{T} = I - \gamma P_\pi$ are also self-adjoint under this inner product:

$$\langle L_{\text{rw}}u, v \rangle_{\mathbf{d}} = \langle u, L_{\text{rw}}v \rangle_{\mathbf{d}}, \quad \langle \mathcal{T}u, v \rangle_{\mathbf{d}} = \langle u, \mathcal{T}v \rangle_{\mathbf{d}}.$$

Thus $\{v_k\}$ forms an orthonormal basis for $\mathbb{R}^{|S|}$ under $\langle \cdot, \cdot \rangle_{\mathbf{d}}$:

$$\langle v_i, v_j \rangle_{\mathbf{d}} = \delta_{ij}$$

Expand V and $\mathbf{r}$ in this basis:

$$V = \sum_{k=1}^{|S|} a_k v_k \quad (34)$$

$$\mathbf{r} = \sum_{k=1}^{|S|} b_k v_k \quad (35)$$

where coefficients are given by:

$$a_k = \langle V, v_k \rangle_{\mathbf{d}}, \quad b_k = \langle \mathbf{r}, v_k \rangle_{\mathbf{d}}$$

Substitute into (33):

$$(I - \gamma P_{\pi}) \left(\sum_{k=1}^{|\mathcal{S}|} a_k v_k \right) = \sum_{k=1}^{|\mathcal{S}|} b_k v_k$$

Using the eigenvalue relation from Lemma 1:

$$\sum_{k=1}^{|\mathcal{S}|} a_k \lambda_k v_k = \sum_{k=1}^{|\mathcal{S}|} b_k v_k$$

By orthogonality of $\{v_k\}$, we equate coefficients:

$$a_k \lambda_k = b_k \quad \text{for each } k$$

Thus:

$$a_k = \frac{b_k}{\lambda_k} = \frac{\langle \mathbf{r}, v_k \rangle_{\mathbf{d}}}{1 - \gamma(1 - \mu_k)}$$

Reconstructing V :

$$V = \sum_{k=1}^{|\mathcal{S}|} \frac{\langle \mathbf{r}, v_k \rangle_{\mathbf{d}}}{1 - \gamma(1 - \mu_k)} v_k$$

□

B. Implementation of VPS-Based Option Discovery

Algorithm 1 Replay-Based VPS Option Discovery

Require: Number of random rewards k , number of options $2k$, buffer size B , VPS learning rates α , β , option Q-learning rate η , discounts γ_V, γ_Q , termination constant N

- 1: **Initialize** replay buffer $\mathcal{D} \leftarrow \emptyset$
- 2: **while** $|\mathcal{D}| < B$ **do**
- 3: Collect transitions (s, a, s') under a random policy and store them in $\mathcal{D}$
- 4: **end while**
- 5: **Stage 1: VPS Estimation**
- 6: Sample $Z \in \mathbb{R}^{|S| \times k}$ with standard Gaussian entries and use the columns of H from the reduced QR decomposition $Z = HC$ as rewards $\{r^{(i)}\}_{i=1}^k$, cf. (18)
- 7: **for each** $i = 1$ to k **do**
- 8: Initialize $V^{(i)}(s)$ and $\varphi^{(i)}(s)$ arbitrarily for all s
- 9: **repeat**
- 10: **for each** sampled transition $(s, a, s') \sim \mathcal{D}$ **do**
- 11: $V^{(i)}(s) \leftarrow V^{(i)}(s) + \alpha[r^{(i)}(s) + \gamma_V V^{(i)}(s') - V^{(i)}(s)]$
- 12: $\varphi^{(i)}(s) \leftarrow \varphi^{(i)}(s) + \beta[(V^{(i)}(s) - V^{(i)}(s'))^2 - \varphi^{(i)}(s)]$
- 13: **end for**
- 14: **until** $V^{(i)}$ and $\varphi^{(i)}$ converge for all s
- 15: **end for**
- 16: **Stage 2: Option Q-Learning (dual directions)**
- 17: **for each** $i = 1$ to k **do**
- 18: **for each** sign $\xi \in \{+1, -1\}$ **do**
- 19: Define intrinsic reward $r_{\text{int}}^{(i, \xi)}(s, s') = \xi(\varphi^{(i)}(s') - \varphi^{(i)}(s))$
- 20: Initialize $Q_{i, \xi}(s, a)$ arbitrarily for all s, a
- 21: **repeat**
- 22: **for each** sampled transition $(s, a, s') \sim \mathcal{D}$ **do**
- 23: $Q_{i, \xi}(s, a) \leftarrow Q_{i, \xi}(s, a) + \eta[r_{\text{int}}^{(i, \xi)}(s, s') + \gamma_Q \max\{0, \max_{a'} Q_{i, \xi}(s', a')\} - Q_{i, \xi}(s, a)]$
- 24: **end for**
- 25: **until** $Q_{i, \xi}$ converges
- 26: Define intra-option policy $\pi_{i, \xi}(s) = \arg \max_a Q_{i, \xi}(s, a)$
- 27: Define termination $\beta_{i, \xi}(s) = \begin{cases} 1, & \text{if } \max_a Q_{i, \xi}(s, a) \leq 0 \\ 1/N, & \text{otherwise} \end{cases}$
- 28: **end for**
- 29: **end for**
- 30: **Return:** $2k$ VPS options $\mathcal{O} = \{(\mathcal{S}, \pi_{i, \xi}, \beta_{i, \xi}) \mid i = 1, \dots, k; \xi = \pm 1\}$

For continuous or high-dimensional settings, RFF rewards from (5) and neural value/VPS losses from Section 4.4 replace the tabular QR rewards and tabular updates. When options are trained, the positive/negative intrinsic rewards and option-learning stage in Algorithm 1 are then applied to these neural VPS estimates. Proposition 1 applies to the online estimator (17) with diminishing, state-wise Robbins–Monro step sizes; it does not provide a convergence guarantee for this fixed-buffer, constant-step-size replay implementation.

C. Experimental Details and Additional Results

C.1. Detailed Experimental Settings

All tabular experiments (MiniGrid-KeyLock, Taxi-v3) in this paper were conducted on an Intel i7-13700HX CPU, while all high-dimensional or continuous experiments (PointMaze-FourRooms, Atari 2600 games) were performed on a single Nvidia RTX 4090 GPU. The detailed experimental settings for each environment are as follows.

MiniGrid-KeyLock

- Environmental Settings

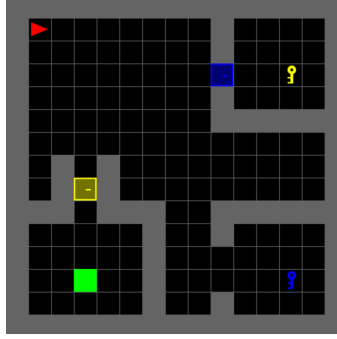


Figure 10. MiniGrid-KeyLock

Observation Space: a tuple of discrete variables (x , y , $direction$, $yellow_door_open$, $blue_door_open$, $yellow_key_on_map$, $blue_key_on_map$), where x and y are the agent position, $direction$ has 4 possible values, each door state is binary (open/closed), and each key state is binary (on the map vs. carried/consumed).

Action Space: 6 discrete actions (up, down, left, right, pick up key, toggle door)

- Option Training phase

Parameter	VPS Option	Eigenoption	Random Option
Intrinsic reward number (k_{base})	4/8/16	4/8/16	4/8/16
Option number	8/16/32	8/16/32	8/16/32
Discount for state-value function (γ_V)	0.999	—	—
Discount for Q-learning (γ_Q)	0.999	0.999	0.999
Learning rate for state-value function (α_V)	0.05	—	—
Learning rate for Q-learning (α_Q)	0.1	0.1	0.1
Buffer episodes (N_{ep})	2000	2000	2000
Episode length (T_{ep})	500	500	500

Table 2. Hyperparameter settings for option training in MiniGrid-KeyLock.

- Exploration Experiments

MiniGrid-KeyLock exploration includes two experiments. (i) **State coverage experiment** (Figure 4(b)) evaluates how efficiently different option sets cover the state space under option-based random walk. (ii) **Random-walk success-rate test** (Figure 14) measures the probability of reaching the goal under option-based random walk. The experimental configurations for these two exploration experiments are summarized in Table 3 and Table 4, respectively.

Parameter	Value
Number of option sets (n_{outer})	5
Runs per option set (inner)	1
Steps per run (max_steps)	2000
Option execution horizon L	15

Table 3. Core parameters for the state coverage experiment in MiniGrid-KeyLock (Figure 4(b)).

Parameter	Value
Number of option sets (n_{outer})	5
Episodes per trial	10
Maximum steps of an episode (max_steps)	200
Option execution horizon L	15

Table 4. Core parameters for the random-walk-success-rate experiment in MiniGrid-KeyLock (Figure 14).

- Reward Collection Experiments

Parameter	Value
Episodes per run	5000
Steps per episode	200
Epsilon-greedy (ϵ)	Annealed from 1 to 0.1
Q-learning learning rate (α)	0.5
Discount factor (γ)	0.99
Option horizon L	15
Runs per option set (n_{inner})	1
Number of option sets (n_{outer})	5
Reward for reaching goal	1.0
Step penalty	-0.01

Table 5. Core parameters for the reward collection experiment in MiniGrid-KeyLock.

Taxi-v3

- Environmental Settings

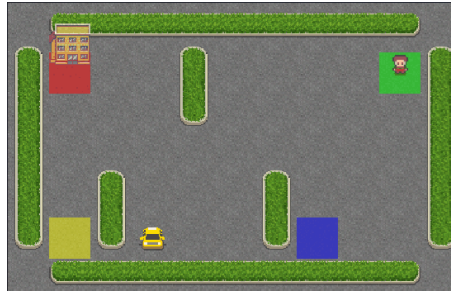


Figure 11. Taxi-v3 Environment

Observation Space: 500 discrete states (Taxi position, passenger location, destination)

Action Space: 6 discrete actions (south, north, east, west, pickup, dropoff)

Reward for successful dropoff: 20

Step penalty: -1

Invalid pickup/dropoff penalty: -10

- Option Training Phase

Parameter	VPS Option	Eigenoption	Random Option
Intrinsic reward number (k_{base})	10	10	10
Option number	20	20	20
Discount for state-value function (γ_V)	0.999	–	–
Discount for Q-learning (γ_Q)	0.999	0.999	0.999
Eligibility trace for TD(λ) of V (λ)	0.9	–	–
Learning rate for all Q-learning (α)	0.05	0.05	0.05
Buffer episodes (N_{ep})	1000	1000	1000
Episode length (T_{ep})	200	200	200
Q-learning update steps (N_{step})	1,000,000	1,000,000	1,000,000

Table 6. Hyperparameter settings for option training in Taxi-v3.

- Random-Walk Task Completion Experiment

Parameter	Value
Number of option sets (n_{outer})	10
Episodes per set	1000
Max steps per episode	200
Option horizon (L)	10
Success criterion	Pickup and dropoff completed within an episode
Action selection	Uniform over primitive actions and options

Table 7. Core parameters for the random walk task completion experiment in Taxi-v3.

- Reward Collection Experiments

In this experiment, Taxi-v3 is treated as an SMDP: the agent follows an ϵ -greedy policy over an augmented action set containing both primitive actions and learned options. During exploration, the exploratory action is sampled uniformly from this augmented action set. When an option is selected, it is executed for at most $L = 10$ primitive steps, and the option-level update uses the cumulative discounted reward collected during execution.

Parameter	Value
Episodes per run	5000
Steps per episode	200
Evaluation interval (episodes)	10
Evaluation trials per interval	10
ϵ -greedy (ϵ)	0.1
Q-learning learning rate (α)	0.1
Discount factor (γ)	0.95
Option horizon (L)	10
Runs per option set (n_{inner})	10
Number of option sets (n_{outer})	10

Table 8. Core parameters for the reward collection experiment in Taxi-v3.

PointMaze-FourRooms

- Environmental Settings

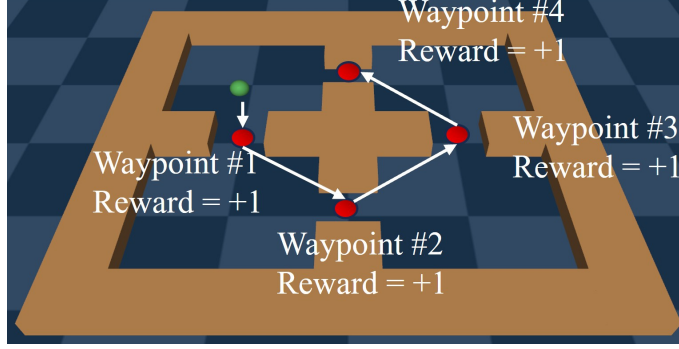


Figure 12. PointMaze-FourRooms environment. The point-mass agent navigates through four rooms connected by narrow passages and receives a sequential waypoint reward. Each waypoint gives reward +1 when it is reached and then activates the next waypoint.

Environment: continuous-control PointMaze-FourRooms with four rooms connected by narrow passages. These passages form geometric bottlenecks, while the sequential waypoint task requires the agent to visit task-relevant locations in order.

Observation Space: continuous point-mass state features containing position and velocity information.

Action Space: two-dimensional continuous velocity-control actions.

Reward: four sequential waypoints are placed in different rooms. Reaching the currently active waypoint gives reward +1 and activates the next waypoint. The downstream episode starts from cell (2, 2), has a maximum horizon of 1000 primitive steps, and terminates successfully after all four waypoints are reached.

- VPS Option Pretraining Phase The main VPS option pretraining parameters are summarized in Table 9.

Parameter	Value
Random-walk transitions	500,000
Episode length	300
Number of RFF rewards	4
RFF frequency scale (σ)	0.001
Value discount (γ_V)	0.999
Value/VPS learning rate	1×10^{-3}
Value/VPS epochs	50 / 50
Value/VPS batch size	256
Number of VPS options	4×2
Option replay buffer size	200,000
Option discount (γ_Q)	0.99
Option learning rate	3×10^{-4}

Table 9. Core VPS option pretraining parameters for PointMaze-FourRooms.

- DIAYN Skill Pretraining Phase The main DIAYN skill pretraining parameters are summarized in Table 10.

Parameter	Value
Number of skills	8
Training steps	500,000
Episode length	300
SAC discount (γ)	0.99
SAC learning rate	3×10^{-4}
Replay buffer size	200,000
Batch size	256
Discriminator learning rate	3×10^{-4}
Discriminator batch size	256

Table 10. Core DIAYN skill pretraining parameters for PointMaze-FourRooms.

- Downstream Waypoint Task and Option Intervention The main downstream task and option-intervention parameters are summarized in Table 11.

Parameter	Value
Waypoint order	$(4, 2) \rightarrow (6, 4) \rightarrow (4, 6) \rightarrow (2, 4)$
Start cell	$(2, 2)$
Episode length	1000
Reward per waypoint	+1
Maximum return	4
Training steps	500,000
Discount factor (γ)	0.95
Learning rate	1×10^{-3}
Replay buffer size	500,000
Batch size	256

Table 11. Downstream PointMaze-FourRooms task and SAC intervention parameters.

All downstream intervention variants use the pretrained skills/options only to guide exploration; the SAC objective remains the sparse external waypoint reward.

Atari 2600 Games

- Environmental Settings

Observation Space: single-frame grayscale images of size 84×84 .

Transition data are collected by an RND-driven exploration policy whose predictor network is trained online during environment interaction (Burda et al., 2019). Eight independent RFF reward probes are used to train eight value/VPS estimators. The Atari experiments evaluate their aggregated VPS score on human-gameplay frames; they do not train or execute option policies.

- Network Structure

The neural networks employed for estimating the value function and VPS in Atari 2600 games share the CNN feature extraction backbone illustrated in Figure 13(a), followed by the estimation head in Figure 13(b). Here, S_t denotes a preprocessed, single-frame grayscale image of size 84×84 ; the feature label $\phi(S_t)$ retained in the architecture graphic denotes the encoder output $z_\omega(S_t)$, not encoder parameters, and the output dimension is $k = 8$ for the eight reward probes.

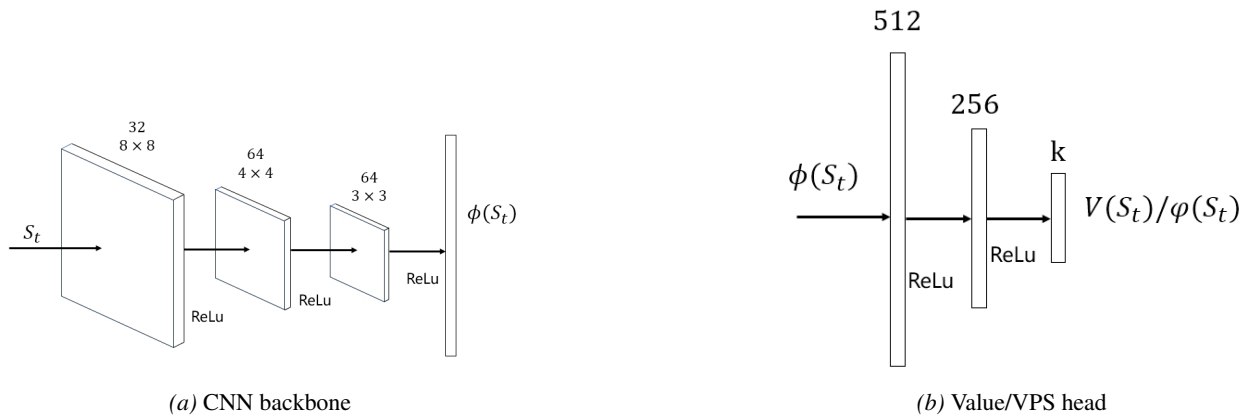


Figure 13. Network architectures for Atari value and VPS estimation.

- Hyperparameter Settings

Parameter	Value
Number of RFF reward probes / VPS estimators	8
Discount for VPS (γ_V)	0.99
Buffer size	500,000
Value training iterations	500,000
VPS training iterations	500,000
Batch size	128
Maximum episode length	500
Value network learning rate	1×10^{-3}
VPS network learning rate	1×10^{-4}
Value network loss function	Smooth L1 loss
VPS network loss function	Smooth L1 loss

Table 12. Hyperparameter settings for value/VPS estimation in Atari 2600.

C.2. Supplementary Experimental Results

MiniGrid-KeyLock Random-Walk Success

This result supplements the state-coverage and Q-learning results by measuring whether option-augmented random walks can complete the full key-door-goal sequence within a fixed horizon. The distribution shows that VPS options improve goal-reaching under random walk, while Eigenoption performance can drop for larger k because higher-frequency Laplacian eigenvectors make options less bottleneck-focused.

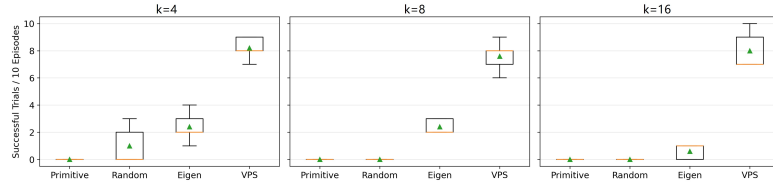


Figure 14. Successful-episode distribution in MiniGrid-KeyLock under option-based random walk with k pairs of options. At each step, the agent uniformly selects among 6 primitive actions and all startable options. We evaluate 5 seeds with 10 episodes per seed (max 200 steps). Box: Q1–Q3; center line: median; triangle: mean; whiskers: $1.5 \times \text{IQR}$.

VPS Distribution in Other MiniGrid Environments

These additional heatmaps show that VPS highlights bottleneck-like regions beyond MiniGrid-Rooms, including doorway and corridor structures in MiniGrid-FourRooms and Maze.

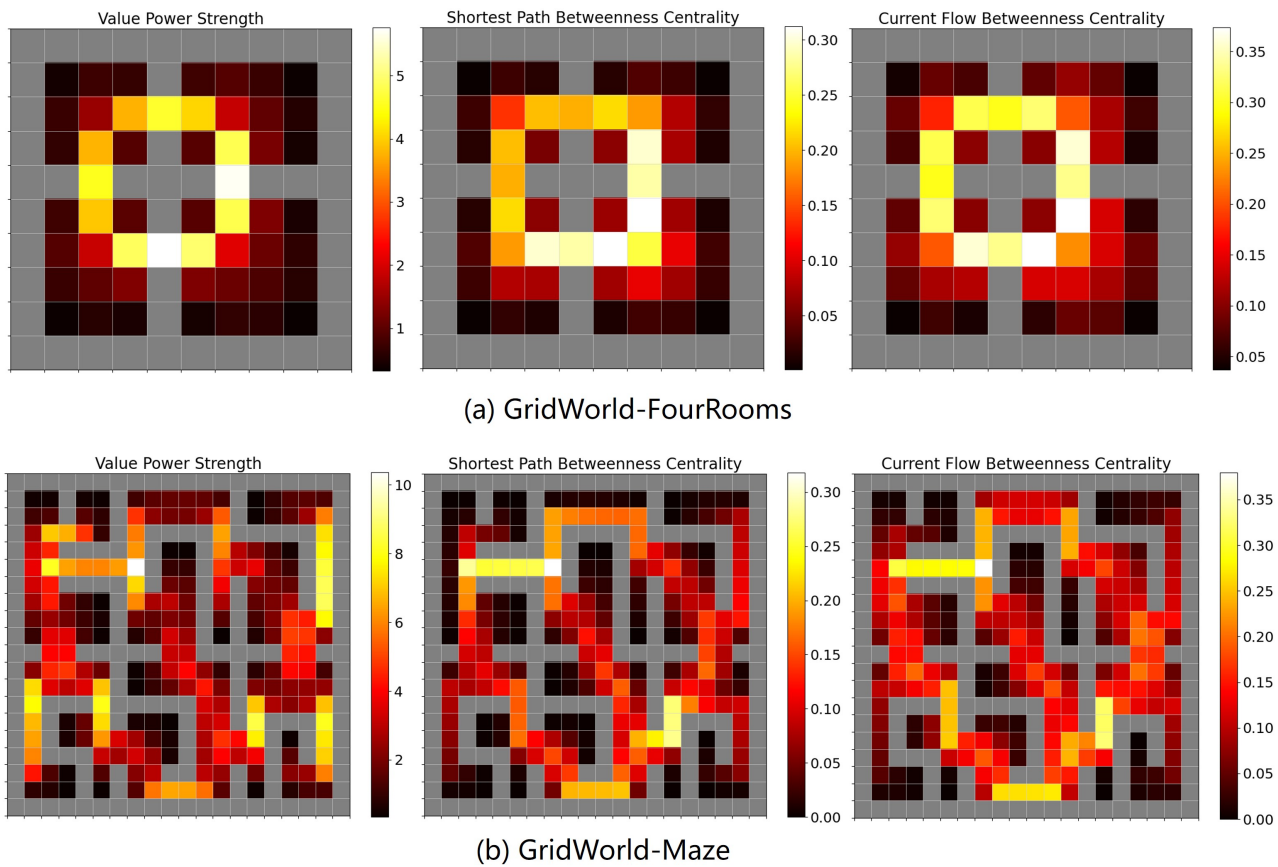


Figure 15. VPS Distribution in MiniGrid-FourRooms and Maze

A Group of 20 VPS Options in MiniGrid-Rooms

This option set illustrates that different VPS-induced intrinsic rewards produce diverse, interpretable behaviors while remaining anchored to bottleneck-related regions.

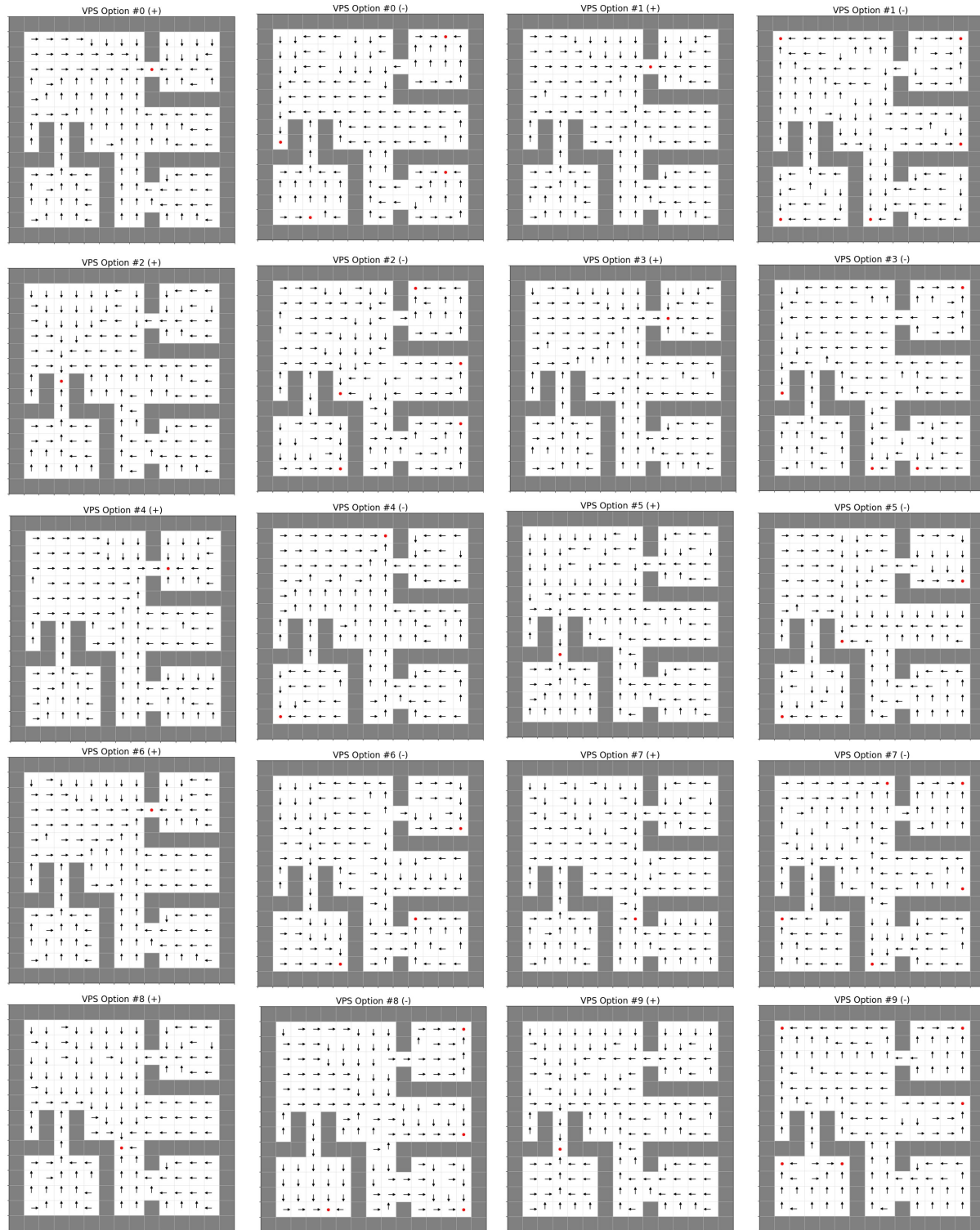


Figure 16. VPS Option Set #0 in MiniGrid-Rooms

DIAYN Rollouts in PointMaze-FourRooms

These qualitative rollouts show reward-free DIAYN skills under the same maze layout used in the downstream PointMaze comparison. They visualize the baseline skill repertoire learned without task rewards, complementing the VPS option rollouts in Figure 7 and the downstream return comparison in Figure 8.

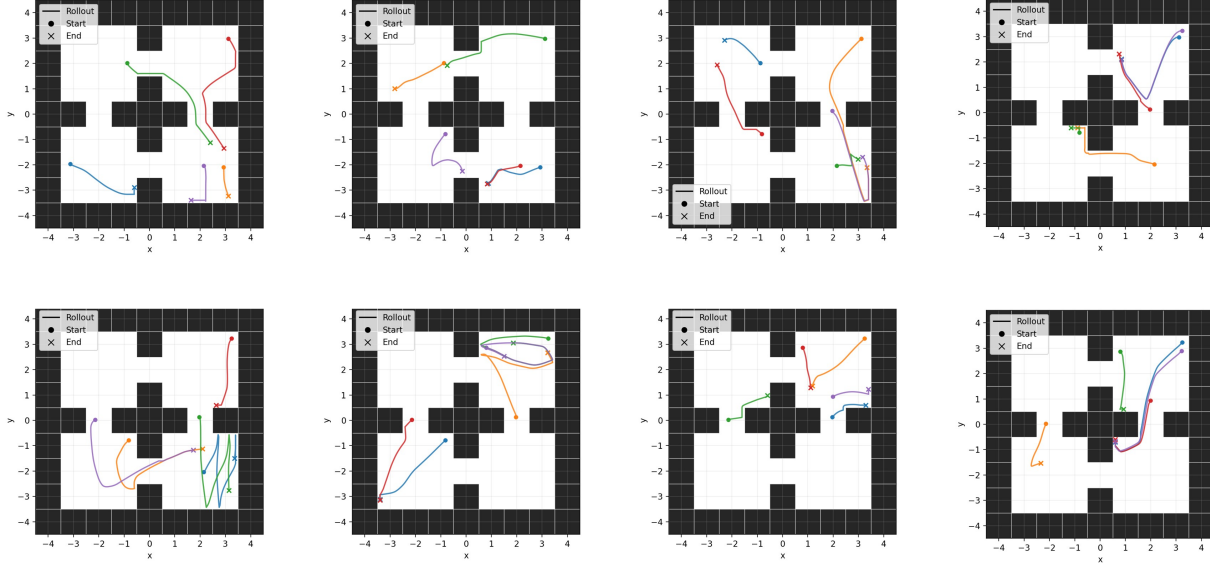


Figure 17. Representative DIAYN skill rollouts in PointMaze-FourRooms. Each panel visualizes trajectories produced by learned reward-free skills from different start states; dots and crosses mark rollout starts and endpoints.

VPS Peaks in Atari-PrivateEye

In ALE/PrivateEye-v5, peaks of the VPS score aggregated across eight independently trained estimators appear around visually and behaviorally salient transitions, indicating states where reward diffusion is strongly concentrated.

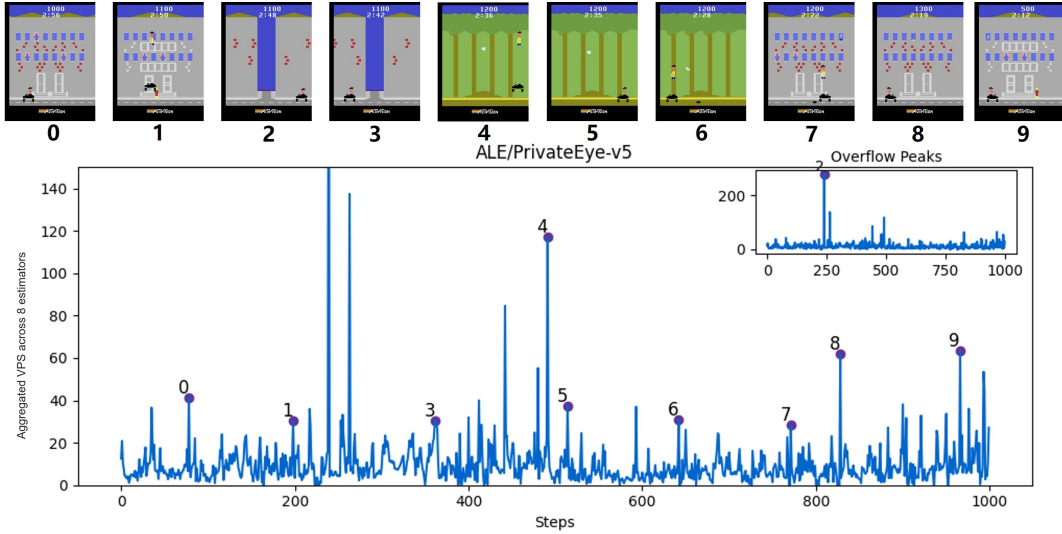


Figure 18. Peaks of the aggregated VPS score in ALE/PrivateEye-v5, evaluated on a human-gameplay trajectory.

VPS Peaks in Atari-MontezumaRevenge

In ALE/MontezumaRevenge-v5, peaks of the VPS score aggregated across eight independently trained estimators emphasize sparse, hard-to-reach visual states that are consistent with bottleneck-like exploration events.

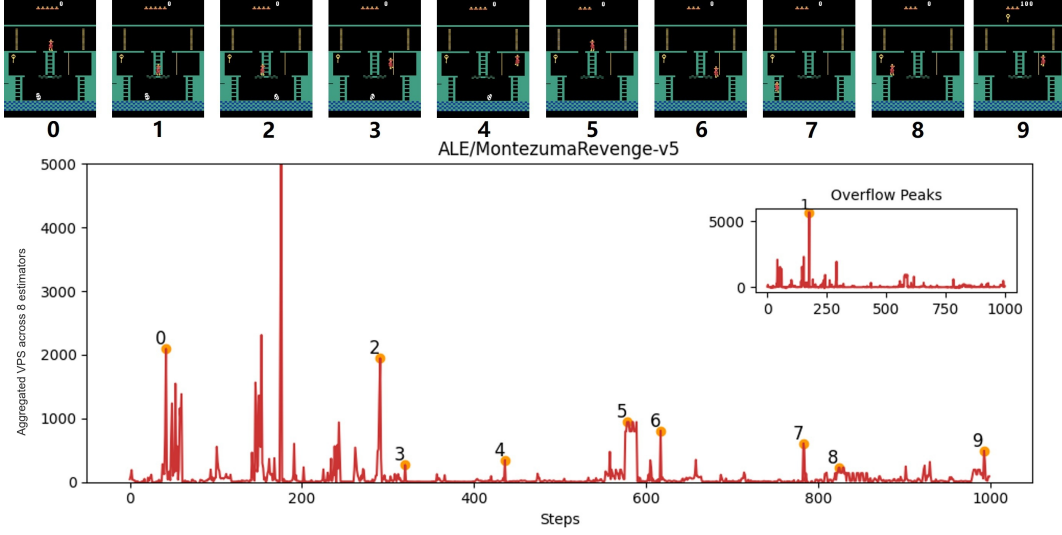


Figure 19. Peaks of the aggregated VPS score in ALE/MontezumaRevenge-v5, evaluated on a human-gameplay trajectory.

VPS Peaks in Atari-Freeway

In ALE/Freeway-v5, peaks of the VPS score aggregated across eight independently trained estimators capture traffic-crossing states where small changes in position can strongly affect future reward propagation.

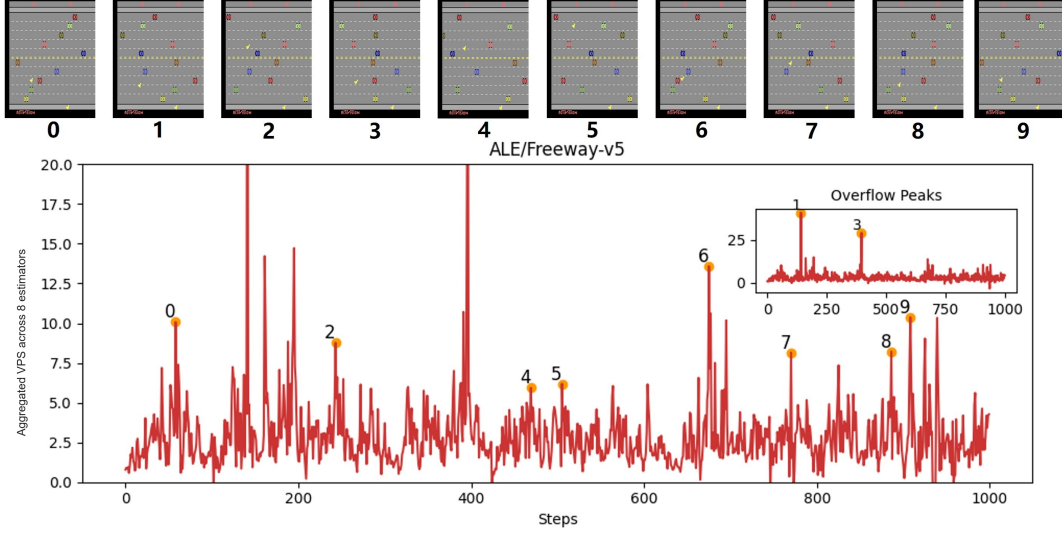


Figure 20. Peaks of the aggregated VPS score in ALE/Freeway-v5, evaluated on a human-gameplay trajectory.